

The Assembled Verb: repertoire, assembly, and the authorship of a verb

Alvaro Rivera

2026-08-27

Abstract

Domain operations already compute, and programs already sequence and coordinate them: in two well-regarded reference systems, the authors' own test suites still exercise their domains once store, bus and test double are removed. This paper is about what neither fact supplies. Executing a sequence consumes capabilities; it does not thereby produce one — before the run there are three capabilities, and after the run there are the same three. The paper reports an arrangement in which assembly constitutes the composition itself as a verb of a subject — one recorded definition whose content is the operations — and measures the difference that makes: performed twice with different arguments, the journal holds one definition and two invocations, the second stating nothing about the verb's constitution. Its recorded constitution is distinguishable by a criterion any record can be asked — how many of the constituting statements does the record contain, and does replaying the record alone bring the outcome about again? — measured across an event-sourced coordinator, durable execution on a third-party engine, a trace, and the constituted verb: only the last both contains and re-performs, and what separates the strongest rival is how much stays ambient in code. Command/query modality is attributed at assembly, per performance, rather than inherited: one body, held fixed, leaves no history exercised as a query and is journaled, replayable, performed as a command. A census of the corpus's 72 named artifacts, under published population and coding rules, shows that in neither system did the plane appear of its own accord; no wider population is claimed. Sixteen neighbour families are conceded by name, and what none of them was found to hold together is the conjunction: a subject-owned record of a definition and its exercises, constituted from repertoires the subject does not own, with modality — including what counts as the subject's history — attributed at the plane of assembly. A prototype of a few hundred lines realizes the same conjunction on a third-party engine, and the record's growth, replay time and throughput are measured against the direct baseline. The paper is analytic in Gregor's (2006, Type I) sense: it prescribes nothing, and its distinctions are stated as propositions over a core operational calculus, with the labs as conformance checks. Assembly turns the use of capabilities into a capability of

a subject — and once the composition has a place of its own, continuity no longer has to be authored into its constituents.

The Assembled Verb: repertoire, assembly, and the authorship of a verb

TL;DR

A(); B(); C(); consumes three capabilities and leaves three. This paper measures an arrangement where assembly *produces* one: the composition becomes a verb of a subject — one definition in the subject’s record, exercised many times by reference — and the paper’s instrument is a question askable of any record, which separates that from a coordinator’s record, a durable-execution history and a trace. Where such a unit is missing, the continuity still has to go somewhere, and shipped software authors it into the pieces: §6 measures those compensations, and a census of 72 named artifacts finds no name for a trajectory spanning subjects. So the consequence the series takes forward is not that composition became first-class, but where continuity may live: once the composition has a place of its own, it no longer has to be authored into its constituents — and the constituents need not know the future wholes in which they will participate.

Dependencies. This paper is part of the *Puppeteer Papers*, a series of self-deposited preprints, and rests on four of them: the actor’s speech and **tell** (Paper 4), whose two directions supply the assembly plane’s own statements in §5.1; the output authority (Paper 8), whose producer-is-not-the-authority geometry §2.4 meets again on the plane of historicity; **Reaction** (Paper 3), whose per-verb consistency contract §2.4’s movement parallels and whose mechanism §8’s corollary draws on; and identity across stagings (Paper 9), whose corpus-selection and anchor-verification discipline this paper follows. The methodological position — what kind of contribution this is, and the boundary against the nearest alternative — is stated in §1.2, beside the evidence it classifies.

1. The Ordinary Ground

Start with the most ordinary thing in programming. A domain offers operations, and a program uses them:

```
A();  
B();  
C();
```

Nothing here is new, and nothing is claimed about it. Ask the equally ordinary question:

What did that program just do?

There are good answers. One can enumerate the operations — it did A, then B, then C. One can put a method around them, and then the program did `CompletePurchase()`:

```
CompletePurchase() {
    A();
    B();
    C();
}
```

One can connect them with events instead of calls, or put a saga or a process manager in charge of the order, or spread them over services. All of these achieve the same thing, which deserves a plain name: **movement**. The operations run, in an order, with values flowing between them.

This paper concedes movement in full, and the concession is not decorative. Two open, independently authored reference systems were taken as a corpus — a widely used commerce reference implementation (.NET Foundation 2026) and a widely cited modular monolith (Grzybek 2024), both selected before any measurement for being well-regarded implementations of the design approach they follow (domain-driven design; Evans 2003). They achieve movement in several of the ways just listed, and they achieve it well. Nothing in this paper improves on how they move, and no engineering incapacity is being solved.

1.1 Two arrangements, side by side

Grant the movement. Now place a second arrangement beside the first:

arrangement one	arrangement two
-----	-----
A();	V := A(); B(); C();
B();	V(x);
C();	V(y);

and ask the question this paper is about:

What exists in the second arrangement that did not exist in the first?

Work through the candidate answers, because each elimination is doing real work.

Not *the sequence* — both arrangements have it. Not *the name* — `CompletePurchase()` can have a better one. Not *the coordination* — both can be coordinated, by any of the corpus’s mechanisms. Not *the result* — both can produce exactly the same effects. Not even *the composed act* — both can execute exactly the same operations, in the same order, with the same values.

What the second arrangement offers — and the paper must now earn rather than assert — is this: **a reusable capability whose constitution is A, B**

and C. In the first arrangement the composition is consumed by the run that performs it. In the second it exists *between* its uses — $V(x)$ and $V(y)$ refer to it — and it remains afterwards. (A method can claim the same on the program-text side, and the claim is not dismissed in passing: §2.5 raises it and §3 decides it.)

before	A	B	C	
assembly	$V := A;$	$B;$	C	
after	A	B	C	+ V

The program of arrangement one *used* three capabilities. Assembly *produced* one. That is not a defect of arrangement one — nothing about it is supposed to produce a capability, and no system in the corpus is failing at a task it undertook. It is a difference in kind, and once it is on the table a second question becomes unavoidable, one level down from the first: **whose capability is V?** A capability, unlike an execution, has to live somewhere. §4 answers that; §2 and §3 first establish that the thing needing an owner exists.

The difference in kind also carries a quiet corollary, stated here and measured in §5: if V can be produced at assembly, then A, B and C did not need to anticipate it. **A repertoire need not know the verbs in which its operations will later participate.**

None of this is hypothetical, and §6 measures it in shipped software. Where the whole has nowhere to live as a unit, the movement is still needed — so it is authored into the pieces: events that carry an aggregate so a handler can fetch it again, correlation carried by a durable queue and a poller, coordinators holding the relation as state of their own. All of it real, all of it buying guarantees its system needs — and all of it work that exists because the composition has no place of its own. So the question underneath every section that follows is not whether the pieces can be tied together — today’s software is the standing proof that they can — but **where the continuity lives**, and what would have to exist for it to live somewhere other than inside the pieces. Stated as a promise, to be cashed at the end: the assembled verb will matter less because it adds one capability than because it gives continuity somewhere to live other than in the capabilities it joins.

The paper’s claim, in one sentence, to be earned over the sections that follow:

Assembly does not merely execute a composition. It makes the composition available as a verb of a subject.

1.2 What is measured, what is conceded, and what is read

Seven bodies of evidence appear and answer different questions. Two are the corpus authors’ artifacts, four are constructed or measured by this paper, and one is not evidence of fact at all; a reader is owed every one of those differences.

	shows	kind
the corpus’s own domain tests	the operations compute on their own	observational
the corpus’s architecture	movement without a produced capability	observational
the substrate labs (§2–§5)	assembly producing one	author-constructed
the long scenario (§5)	the plane made visible across repertoires	author-constructed
the third-party rows and the prototype (§3.2)	the criterion’s cells, and the plane itself, on engines not this paper’s	author-driven, third-party substrate
the record’s prices (§8)	what the arrangement materially costs, against the direct baseline	author-measured
the calculus (Appendix C)	the separations as propositions with proofs	formal — construction, not evidence of fact

The constructed evidence proves realizability and nothing about how either corpus system was designed; no count from it is offered as evidence about their authors’ choices. The corpus was selected on stated criteria before it was read, and the census of §6 was run after the selection. No population-level generalisation is claimed anywhere.

The record’s cost is measured and nothing is ranked by it: §8 prices growth, replay and throughput against the direct baseline and claims no advantage anywhere.

Methodologically the paper is an analytic theory contribution in the sense of Gregor’s (2006) theory for analyzing (Type I): capability, exercise and constitution are constructs by which arrangements may be described and compared, and the paper offers no prescription — its instrument is the question of §3, askable of a system already built. The distinctions themselves are stated operationally in Appendix C, over a core calculus small enough that each separation is a proposition with a constructive proof, and the labs double as its conformance checks. The labs of Appendix A are accordingly an existence proof of realizability — on two engines, one of them third-party — plus a measurement of the record’s material cost (§8); no advantage claim is made anywhere, and no design-science evaluation of benefit is attempted. That an artifact is built and measured invites the nearest alternative classification, so the boundary is stated checkably rather than trusted: a design-science contribution (Gregor’s Type V; Hevner et al. 2004; Peffers et al. 2007) states a problem, derives objectives of a solution, and evaluates the artifact’s utility against them, and this paper does none of the three, on purpose. What the labs evidence is therefore analytic, not

evaluative: that the constructs denote — the category is instantiable, on more than one substrate — and what its instances measurably are. A reader who finds a utility claim anywhere in this paper has found an error.

The argument underneath runs in four steps, and the sections divide between *walking* it and *holding the ground it crosses*. §2 demonstrates the mechanism: V exists — defined once, exercised by reference, replayed from the record. §4 locates it: V belongs to none of the pieces, so a place other than the pieces exists. §5 cashes the deduction: two repertoires that never met participate in one verb of seventeen operations, unmodified — anticipation is not required. §6 shows the condition the corpus lives under where no such place exists: continuity authored into the pieces, measured. The rest holds ground: §3 verifies that the unit really resides where §2 put it; §2.4 establishes the modality property the conjunction of §7 needs; §4.2–§4.3 bound the subject’s position; §7 walks sixteen neighbour families. Necessary — the category would not survive scrutiny without them — and deliberately not the spine.

And one term is used before it is defined, deliberately. *Subject* carries, until §4, only its deflated sense — the thing an operation is attributable to. The strong sense is not in use until it is earned.

2. Assembly Produces a Capability

2.1 What the substrate provides

Two ordinary things. An **actor** (Hewitt, Bishop and Steiger 1973) holds state and is entered by performing acts on it. A **journal** records each act as it is performed. Neither idea is this paper’s, and no novelty is claimed for either.

One mechanical property of the record is the whole of what this section turns on, and it can be stated without any claim about what an entry *is*: **the record preserves enough of each performed operation that replay invokes it again**. Not enough to describe the operation, and not an outcome from which the operation could be inferred — enough to invoke it. The property is checkable by running the replay, and it is visible in the record’s two shapes: a definition entry carries the statement text itself (`EventData.cs:49-52`), an invocation entry carries an integer identity and an argument string and nothing else (`EventData.cs:34-37`), and rehydration reads them back through one entry point (`ActorHandler.cs:3929`).

The domain assemblies used throughout are the corpus’s shipped binaries, unmodified, referenced as prebuilt binaries by a host their authors never anticipated. Nothing was edited to make any of this fit.

2.2 The entry

A composed verb is performed once — five domain operations and one register entry of the assembler’s own. This is what the journal contains afterwards, printed from it rather than transcribed:

```

define action 1 (payerId:string, country:string, period:string, amount:decimal,
                currency:string, street:string, city:string, state:string,
                zip:string, userId:string, userName:string, card:string,
                cvv:string, holder:string, sku:int, itemName:string,
                itemPrice:decimal, key:string) as
    payment    = payments.Buy(payerId, country, period, amount, currency);
    payment.MarkAsPaid();
    address    = Address(street, city, state, country, zip);
    welcomeKit = Order(userId, userName, address, 1, card, cvv, holder,
                      Now.AddYears(1), Null, Null);
    welcomeKit.AddOrderItem(sku, itemName, itemPrice, 0.0, '', 1);
    held.Keep(key, payment);
end;

```

Three facts about that entry, each about the record rather than an interpretation of it.

Its content is the composition. The entry does not merely correlate the operations, index them, or summarise their outcomes. It preserves the statements that constitute the definition, in their performed order, together with the values that flow among them.

The definition is not authored by the developer: the runtime emits it when a composed body is first performed (`DefineActionStatement.cs:25`).

It has no business name. The entry is `action 1` — an integer, because actions in this substrate carry no name at all: the invocation record has fields for an identity and an argument string, and no field a name could occupy (`EventData.cs:34-37`). It could as easily be `action 7` and everything in this section would hold. Whatever makes this entry one unit, naming is not it:

what makes the composition one unit	one definition an ordered set of operations the values flowing among them
-------------------------------------	---

what its status in the history rests on	the execution is attributed to the actor replay re-performs that definition
---	--

Replaying it re-performs the operations. Reading the entry back invokes the same statements, on the same repertoires, with the values the entry carries.

2.3 Reuse — the measurement that separates a capability from a receipt

Everything above is compatible with a weaker reading: the record holds a *receipt* of something that happened once. A capability is more than a receipt. **The test of a capability is reuse: it can be drawn on again without being restated.**

So the measurement (Appendix A: `TheCompositionBecomesACapabilityLab.cs`): the same composed verb is performed **twice**, with different arguments — different payer, amount, item, key. Then the journal is read, and then a fresh actor is built over it.

times the composed verb was performed	2
definitions of it in the journal	1
state after the live run	both purchases held
state after replaying the journal	both purchases held

The second use did not restate the composition. It **invoked** it: what travelled was the verb’s identity and the new arguments. And a fresh actor over the same journal replays both invocations from the one recorded definition.

The seam where this happens is small enough to read (`ActorHandler.cs:2241-2255`): a body whose authored form is already known is journaled as an *invocation* of the existing action, identity plus fresh arguments; a body not yet known is journaled once as its *definition* and assigned the next integer identity. The branch is the mechanical difference between a capability and a copy.

An inline sequence can of course be executed twice — by writing it twice. What it cannot do is exist *between* the two executions as a thing the second one refers to. That existence-between-uses is what the figure of §1.1 asserts with its + V, and it is now exhibited in an inspectable record rather than asserted as notation:

Before assembly there were N capabilities. After assembly there are
N + 1, and the journal can be inspected for the +1.

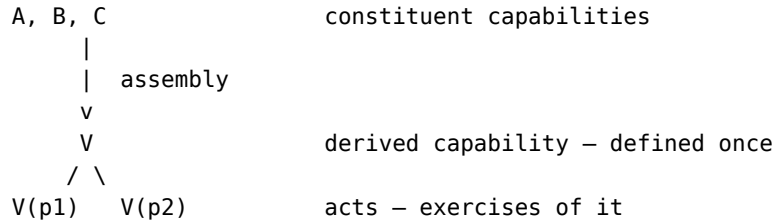
The status of that sentence has to be stated exactly. *Capability* is operationalized here by the reuse test, and the arrangement measured implements that operationalization — so the N + 1 is not the confirmation of a hypothesis. It is two smaller things, each worth having. It is a **demonstration that the definition is realizable**: a record can hold the definition/exercise separation as data, which none of the ancestors examined in §7.1 establishes for a *subject’s own record*. And it is a **conformance check that could have failed**: had the second use re-emitted a second definition, the seam of `ActorHandler.cs:2241-2255` would have been broken and the lab would have counted two. What keeps the construct from being private to this paper is §7.1: *defined once, exercised many times* is not coined here — macro-operators, options and workflow cases hold it independently — and what this section adds to that inherited construct is only *where it lives*.

The measurement licenses a deduction stronger than the count. Its premises are measurements of this arrangement; only its *form* names no substrate:

1. A, B and C are available capabilities.
2. Assembly defines $V := A; B; C$.
3. V is invoked later, with values that did not exist when V was defined.
4. Two distinct invocations refer to the same V.

5. Replay preserves both the definition and the distinction between its exercises.

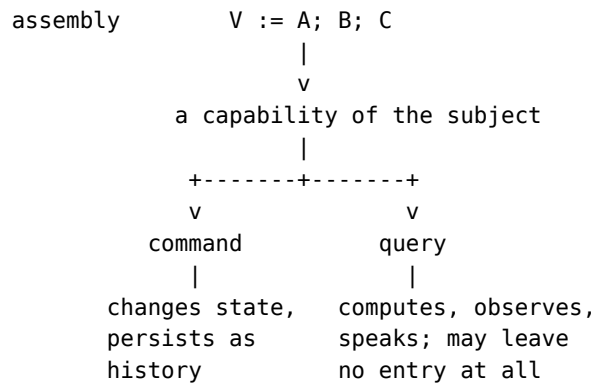
Therefore **V cannot be identified with any of its executions** — they differ and it does not. And V cannot be identified with A, B and C executing either: that is its *constitution*, not V itself. What remains is a structure with three levels, none reducible to the others:



Defining a capability is not exercising it. The journal of this section holds that separation as data: one definition, two exercises, and replay reconstructing both from the one. The deduction establishes what holds of this arrangement, measured — not what must hold of arrangements generally.

2.4 The capability is above its modalities

One narrowing has to be refused before it settles in, because the measurement above invites it: **the assembled verb is not defined by the journal.** What §2.2 and §2.3 exhibit is the *journalized command* modality of its exercises — the one where performances change state and persist. It is the modality where the definition/exercise separation can be shown as data, which is why it was measured first. It is not the definition of the term.



A verb exercised only as a query — operations of several repertoires read, a value computed that none of them holds, and the result pronounced — is, on the definition this section arrives at, as much an assembled verb as §2.2's. Its constitution may be *richer*: it can relate results no repertoire can relate. What differs is the modality of its exercises: a query executes under a read lock and

does not persist to the journal (`ActorHandler.cs:2954-2969`), so the reuse measurement of §2.3 cannot be run on them. That bounds the *evidence*, not the *category*.

And the modality is not read off the operation — nor fixed on the verb: it is **attributed by the assembler, per performance**, and the record follows the attribution rather than the operation’s effect. Measured (Appendix A: `WhoDecidesWhatCountsAsHistoryLab.cs`), with everything but the attribution held fixed — one body, one verb under §2.3’s seam, three performances: exercised as a query it leaves the journal head unmoved; performed as a command the act is journaled and a fresh subject replays it from the record alone; queried again after the command, the head stays. Had modality been the verb’s, the second and third performances could not have differed. The domain computes the same value each time and never changes.

journal head after setup	2	
one body exercised as a QUERY	2	unchanged
the same body performed as a COMMAND	3	advanced
the same body queried again	3	unchanged
after replaying the journal from scratch	3	

Nothing in the operation distinguishes the two runs. What distinguishes them is a decision that belongs to the party constituting the verb: **whether this performance counts as the subject’s history**. Disclosing a value and looking at a value can be the same computation and different acts — a subject for whom the disclosure itself must stand as fact constitutes it as a command, and the effect on the domain is identical: none. Two subjects assembled over the same repertoire may therefore have different histories without the repertoire changing.

Practice already knows that a disclosure can be a fact. Access audit logging records reads precisely because the reading itself must stand — the health and payment-card regimes require it of systems handling protected data (HIPAA’s audit controls, 45 C.F.R. § 164.312(b); PCI DSS Requirement 10) — so *journaling a read* is not the observation here, and the section would be conceding its oldest neighbour by implying otherwise. Two separations hold, and both are checkable. **Where the decision lives:** an audit regime fixes the recording decision once, for a category of operations — all access to the protected data is logged — and implements it in the domain or its infrastructure: an interceptor, a middleware, a logging call inside the method. Here the attribution is per performance, taken at the plane of assembly, and the repertoire is untouched — the same operation, under the same verb, enters one act as a query and the next as a command, which no category-wide rule expresses. **Where the fact lands:** an audit log is a second record, kept beside the state the system runs on and consulted apart from it. Here the journaled disclosure enters the subject’s one history — the record the subject is reconstructed from — and replays with it, as the measurement above shows. The kinship strengthens the observation rather than threatening it: what a regime must bolt on beside the domain,

category-wide, the plane attributes verb by verb, editing no one.

What command–query separation (Meyer 1997) separates is therefore real, but it lives one level up from where it is usually placed: it is a property of the subject’s acts, attributed at assembly per performance, not an intrinsic property of the domain’s operations. An operation that changes nothing can constitute a command, because what the attribution governs is historicity rather than effect.

This is a geometry of authority the series has met before, twice. The output finding was that the producer of a value is not the authority on its destination; here, the author of an operation is not thereby the authority on the historicity of the acts that will incorporate it. And the movement of the audit-logging contrast — a property fixed once, system-wide, becoming a decision attributed at a named plane — is the movement the series made for eventual consistency, where a system-wide contract became a per-verb list the assembler wrote by hand; here the grain is finer still, one attribution per exercise. Consistency there, historicity here: the same register, a decision habitually fused into the system, found separately attributable — there verb by verb, here act by act. And where persistence lives inside a domain, its author necessarily takes both decisions at once — what the domain means, and what survives as history. Neither decision is wrong there. What the measurement shows is only that they are separable, and where the second one can live.

So the definition, stated at the level it actually lives:

An assembled verb is a capability of a subject, constituted from operations available to assembly. Command and query are modalities the assembler attributes to its exercises. History is what follows from that attribution — which of the subject’s performances count as its history — not what makes the verb a verb.

Four things, then, that are not synonyms:

domain	what can be computed
assembly	what the subject can do
performance	what the subject does
history	which of its performances count as its history

If the term had been reduced to the journaled command, *assembled act* would have been the honest name. That the query enters naturally is what confirms *verb* was the right abstraction: a repertoire holds what a subject can do, and what a subject can do includes asking and saying, not only changing.

And the four lines are not left as a table. Appendix C states them as four projections of one small operational object — a core calculus in which assembly, exercise, modality and replay are rules — and shows by witness pairs that none of the projections determines the others. The table is the claim; the calculus is where the claim is a property.

2.5 The objection this does not yet answer

A reader should still be unconvinced, on one specific ground: `CompletePurchase()` also exists between two executions — in the program text. Call the method twice and nothing is restated either. Why is **action 1** not simply a method that happens to live in a log?

That objection deserves an instrument rather than a paragraph, and it gets one.

3. The Criterion

Call the composed work a verb performs *the whole*, for this section's purposes. Two other arrangements produce a record that also refers to the whole, and one of them gives it a better name than this one has:

COORDINATION in two measured shapes. An event-sourced process manager, with its own identity, state and stream — it may emit `PurchaseCompleted`, and its name, lifecycle and durability are conceded in full. And a durable-execution orchestration, run on a third-party engine (§7.2), whose record is whatever that engine writes.

DESCRIPTION a trace, in the distributed-tracing sense (Sigelman et al. 2010). Its root span is called `CompletePurchase` and every statement appears beneath it. It records the grouping faithfully.

CONSTITUTION the entry of §2.2. It is called ``action 1``.

The criterion owed here must be askable of any system — including one that has never heard of this substrate. A criterion of the form *is it a journal entry of the kind §2 describes* would answer nothing.

3.1 The question

Take the record that claims to be the whole. Ask two things of it, and nothing else:

(a) **Content.** How many of the statements that constitute the whole does it contain?

(b) **Replay.** Hold the arrangement's engine and the repertoires fixed, and supply nothing else: does **the record alone** determine a replay after which the whole's outcome exists again?

Both are answerable of any record by inspection and by running it, and both are **total**. (b) quantifies over the arrangement's own replay semantics, so an arrangement that supplies no replay operation answers *no* — vacuously, because nothing exists that could bring the outcome about — rather than falling outside the question. And *alone* is load-bearing: it constrains the replay's **input**, not its

mechanics. A replay that reads anything beyond the record itself — beyond the engine that runs it and the repertoires it invokes — is not a replay of the record alone; in particular, a replay that reads a definition kept outside the record, as compiled code or an ambient program, leaves the outcome determined by the pair rather than by the record, and the same record means something else when the program changes (Appendix C states the distinction as a theorem: Proposition 5). The fixing of engine and repertoires is itself a relativization, carried as a limit in §8: what the record closes over is the composition, not the operations. Without the clause, (b) would credit to the record what an arrangement’s own recovery guarantees — durable execution does bring its orchestration’s result back, in its own operation — and the question would stop separating anything; the clause is what makes (b) a question about the *record* rather than about the arrangement’s uptime. Neither question asks what the record is called, which tool produced it, or what its author intended.

And what the two questions classify is a *record*. The criterion is therefore a criterion for **recorded constitution** — not for the assembled verb as such — and the narrowing is load-bearing rather than a concession. §2.4 already met its reason: a verb whose exercises are queries deposits nothing, so no record exists for this section to ask anything of, and the verb is no less a verb. Three levels nest, each strictly wider than the next:

assembled verb	§2.4's definition –
queries included	
▷ journaled assembled verb	the exercises that persist
▷ record satisfying this criterion	recorded constitution –
what §3 decides	

§3 decides the innermost level, and only there is it decisive. Its verdicts are *evidence* for the outer levels — §2.3’s reuse and §2.4’s attribution read the same record — but the category is defined in §2.4 and owned in §4, and neither burden is carried here. A criterion that tried to decide verbhood directly would have to see performances that never enter a record, and would stop being askable of arbitrary records — which is the property that makes it an instrument.

What counts as the outcome existing again. Outcome identity is taken *up to fresh values*: two outcomes are the same when they differ at most by a consistent renaming of identifiers minted during the run rather than supplied to it — the standard move on fresh names. The tolerance is principled, not convenient — and Appendix C proves it *necessary*: under the host axiom, minting is fresh by construction, so exact identity is not available and a bijection on minted names is (Proposition 4). Any criterion that rewards *re-performing* must state one, because re-performance re-mints whatever was minted; a criterion demanding bit-identity would hand the victory to arrangements that never re-perform and declare every re-performing system a failure. The tolerance also rewards nothing that does not re-perform — it only relates outcomes that exist again. And it is not free for this paper: §4.3 measures the value that forces it, in this paper’s own flagship entry, and §8 carries it as a limit. Without this

clause the instrument would fail the paper’s own constitution, which is why it is stated here, before the criterion is used.

3.2 The measurement

The same work — six statements — performed under all arrangements, over the same two prebuilt domain assemblies (Appendix A: `TheCriterionForConstitutionLab.cs` with `ProcessManagerHarness.cs`; `TheEventSourcedCoordinatorOnAThirdPartyEngineLab.cs`; `TheCoordinationArmOnAThirdPartyEngineLab.cs`; `TheConstitutionPrototypedOnAThirdPartyEngineLab.cs`).

The provenance of each coordination row is declared, because a baseline the author configures is not a class. Both coordination rows are measured on third-party engines. The event-sourced shape runs as an Orleans journaled grain — the grain journals its own transitions, reconstructs from that journal, and the operations it coordinates are other grains’ acts — with the record read back through the framework’s public surface; the author-built harness is kept beside it as the corpus-mirror it always was, since the corpus’s own sagas have exactly this shape (§6). The durable-execution shape — the class at its strongest — is measured on the Durable Task Framework, the engine underneath Azure Durable Functions, with the history read through that framework’s public dispatch middleware. In both, the record counted is the engine’s own.

And the constitution row is not left as the only row realized solely on this paper’s engine, because a single implementation cannot separate two readings — *the assembly plane is a realizable arrangement* and *one framework has this feature*. The arrangement is therefore prototyped, in a few hundred lines, on the coordination row’s own engine: a journaled grain whose log holds `def` and `act` entries and whose state transition re-executes the definition’s statements against the same repertoires. The substrate is held constant and only the arrangement varies, and one third-party engine realizes both cells — what decides the cell is what the journal holds and what the fold does with it. The prototype reproduces the row’s every property on a three-statement verb over the same repertoires: one definition under two exercises, the second stating nothing of the constitution; a query performance that executes and leaves the journal unmoved; reconstruction that re-executes every journaled statement from the record alone; and the domain-minted identity differing per replay — §4.3’s boundary, met unchanged on the second engine.

arrangement	names the whole	(a) statements in the record	(b) the record alone replays into the whole
coordination — event-sourced process manager (third-party engine; corpus-mirror harness agrees)	yes	0 — its own journal holds 6 transitions, none an act's statement	no
coordination — durable execution (third-party engine)	yes	6	no
description	yes	6	no — nothing to replay
constitution	no	6	yes — up to fresh values (§3.1)

Naming decides nothing, and it runs the wrong way. The arrangements the last row must be distinguished from are the ones with the business names; the constituted verb is an integer. If naming carried the property, the other rows would have it.

(a) separates only the first shape. The event-sourced coordinator's own journal holds its transitions and no operation: which step it reached, what it awaits — measured on Orleans, six progress marks and zero operations, the operations living in other subjects' acts. The durable-execution record, by contrast, holds all six invocations — names, inputs, results. Content cannot tell it from the description row or from constitution.

(b) separates the rest, and it separates them differently. The trace answers *no* vacuously: it supplies no replay operation, so no replay of it exists after which the outcome could obtain. The durable-execution record has a replay, and in the model's own operation the orchestration recovers and completes — the row's *no* does not deny what the model guarantees. What the row scores is (b)'s input clause: that replay re-executes **the orchestrator's code** against the recorded history — a definition living outside the record — so its outcome is determined by the pair, not by the record alone. Measured: the orchestrator body ran in seven episodes, six of them replays, while every activity executed exactly once; the record alone re-performed nothing, and that is the model's own determinism contract. The constitution row's entry is read back, and reading it back performs the six statements again — nothing outside the record but

the engine and the repertoires.

The input clause is not a footnote the strongest row forced; it is the half of the criterion that decides that row: **where the definition lives**. In durable execution the composition is compiled code, and the history holds exercises and outcomes that only that code can re-walk (§7.2). In the constitution row the definition is itself an entry in the record, which is why — with the repertoires held fixed, as (b) fixes them — the record alone suffices to re-perform. The difference from the strongest coordination is in what stays ambient: there, the composition and the operations; here, the operations only.

Only the conjunction classifies:

- (a) alone description, constitution and durable coordination are indistinguishable
- (b) alone the other three fail it, each for a different reason
- (a) and (b) exactly one arrangement contains the statements and re-performs them

Coordination preserves relations among acts — or, in its durable form, the invocations and their outcomes. Description preserves an account of a grouping. Constitution preserves the grouping as something that can be performed again.

3.3 The method objection, answered

Now §2.5's objection can be put to the instrument. A method with exactly the same body — even the same six statements — leaves what behind after it runs?

method extraction	source: CompletePurchase = [a, b, c]
	history: a, b, c

the arrangement here	source: action 1 = [a, b, c]
	history: action 1 := [a, b, c]

Execution of a method does not by itself preserve its lexical grouping; unless something separately records that grouping, the history contains whatever its constituent operations leave behind. Something separate *can* be added — that is the description row, and it was measured rather than dismissed. What the third row has is different in kind:

**Lexical composition disappears when the program runs.
Constitutive composition survives into the record.**

And note the control hiding in the identifiers: the method is the one with the business name, and it is the arrangement that loses the composition. What differs is not what the composition is called but whether it is still there after the run — still there in the sense the criterion makes precise: containing its statements, and re-performing them when replayed.

3.4 What the criterion does not decide

It does not say a description could not be made replayable. It could — a trace enriched until its record suffices to re-invoke the operations would be a journal, and the criterion would classify it as constitution, which is the right answer. The criterion is indifferent to tools and names; it asks what replaying a record does.

It does not rank the three arrangements. They answer different questions: what follows this act, what happened during it, what constitutes it.

Nor are its two questions without ancestry of their own. The provenance literature has long distinguished prospective from retrospective provenance and asked when a captured record is re-executable (Freire et al. 2008; Moreau and Missier 2013). The kinship is acknowledged rather than annexed: what §3 adds is not either question but their use as a *joint* criterion over one record, whose conjunction — and only the conjunction — separates the three arrangements above.

And it classifies *records*, so its reach is the modalities that leave one. A query exercise of an assembled verb may leave no record at all, and the criterion is silent about it — which bounds the criterion, not the verb (§2.4).

3.5 What kind of instrument this is

One question has to be answered explicitly: is the classification empirical at all? The concession of §3.4 — a description enriched until re-invocable would classify as constitution — shows that the classification is **by construction**. *Constitution*, in this paper, names the conjunction; no observation falsifies a definition, and the paper says so plainly rather than dressing a definition as a discovery. This is the posture §1.2 declared, applied where it bites: the criterion is a Gregor Type I construct, a defined, observable property by which arrangements are described and compared.

What is empirical — and falsifiable — is everything the definition is then put to:

- **each cell of the table.** Every row re-runs from Appendix A, both third-party rows run on their engines' own semantics, and the constitution arrangement additionally runs as a prototype on one of those engines (§3.2), so no cell rests solely on this paper's engine. A Durable Task orchestration whose record, with the orchestrator's code changed, still replayed to the same outcome would overturn its row — the record alone would be deciding; an Orleans coordinator whose journal carried the operations would overturn its.
- **the separations.** That the shipped arrangements occupy distinct cells was not derivable from the definition. The durable row's 6 could not have been read off the definition; it could have come out otherwise, and for one arrangement family it did.

- **the constitution row itself, which is not satisfied by fiat.** Strictly — without §3.1’s fresh-value clause — the flagship entry of §2.2 fails (b): a domain-minted identity does not come about again on replay (§4.3). The equivalence is therefore not a convenience of this paper’s arm; it is *forced* by a measurement against that arm, declared before the criterion is used, and carried as a limit in §8. And the row stays falsifiable under it: a replay that missed the outcome even up to fresh renaming — a lost entry, a wrong count — would overturn it.

So §3 defines a property and measures where arrangements fall under it. The definition is the instrument; the measurements, the separations, and the one partial failure are the findings. And the two halves are kept formally apart: Appendix C restates the definitional half as propositions over a core calculus — so what is by-construction is exactly the part that admits proof — while the labs of Appendix A carry the empirical half, one conformance check per proposition.

And it does not yet say whose verb *V* is. That is the next section, and it is a separate question: a record can hold a composition as one replayable unit without anyone having asked to whom the unit belongs.

4. The Subject

4.1 To whom the verb belongs

The entry of §2.2 is not floating in a store. It is a record **of one actor’s history**: the execution is attributed to the actor that performed it, replay reconstructs that actor from its own record, and the reuse of §2.3 is that same actor drawing on its own verb a second time.

So the capability produced by assembly has an owner, and the owner is none of the ingredients. *V* does not belong to the repertoire that declares *A*, nor to *B*’s, nor to *C*’s — those repertoires know nothing of *V*, and §5 measures just how little. *V* belongs to the repertoire of the subject that performed the assembly.

Which gives the term its two faces, and both are already in the data of §2:

repertoire	history
-----	-----
<i>V</i>	<i>V</i> (<i>p</i> 1)
	<i>V</i> (<i>p</i> 2)

As repertoire, *V* is a **capability** — what the subject *can do*. As performance, *V*(*p*) is an **exercise** of it — what the subject *did*. The two columns answer different questions, and for the journaled modality the record preserves precisely the relation between them: each entry on the right names the definition on the left. For other modalities the right-hand column may be empty and the left-hand one no less real — a query verb is exercised without depositing anything (§2.4). That is what the paper’s title means:

An assembled verb is a capability of a subject, constituted from operations available to assembly (§2.4) — a whole that is the verb of that subject, rather than a coordination among the acts of its parts. The term claims no mechanism. §2 showed one realization; §3 gave the external test; this section supplied the owner — and with the owner, §1.1’s argument gains its location: a place other than the pieces exists. §5 cashes what the place buys (the pieces need not anticipate), and §6 measures what the corpus pays where no such place is.

A sentence held back until here, because before §2 and §3 it would have been a metaphor and would have smuggled in subject, voice and attribution before any of them were shown:

There was no speaker among the pieces for whom the sentence was first-person speech.

The pieces were never missing speech of their own — each names every transition it is entitled to name. What was missing was a speaker for what spanned them, and the actor of §2 is that speaker in the only sense this paper uses: the composition is in *its* record, attributed to *it*, replayable as *its* act.

And “the only sense this paper uses” is a boundary, carried as a limit in §8. What the record establishes is **ownership**: the definition and its exercises are in one actor’s record, attributed to it, and replay reconstructs that actor from that record. **Subjecthood is not derived from constitution.** The same data admits a flatter description — *an actor instance owning a dynamically extended executable vocabulary* — and every measurement in this paper survives it unchanged; what the richer vocabulary adds is names for the relations measured — owner, exercise, history — not further claims. That the owner is a *subject* is the actor model’s frame and the series’ prior finding that identity precedes staging: assumed here, and argued elsewhere.

4.2 What the subject’s position is, and is not

Two misreadings have to be refused.

The subject does not know more about any repertoire than the repertoire does. Its additional knowledge is about the relation among the pieces, not about the interior of any piece. It can know that one value participates in two operations, because it occupies the only position where both are present; it cannot decide what that value means inside either. (Both halves are exhibited concretely in §5: the co-binding scope of §5.1, and §5.2’s unstatable proposition.) Its reach has **width, and not depth** — reach rather than authority, because a runtime capability and a permission are different things and must not be fused:

The assembler can say more than either repertoire, because it sees both. It cannot say more about either repertoire than that repertoire

permits.

Composition gives it no operation a repertoire did not already provide: no added method, no relaxed invariant, no authorship over decisions made inside an invoked operation. **It can say a sentence no part can say, and cannot correct a single word of the sentences the parts can say.**

And the subject’s position was not granted by the pieces’ authors. The repertoires here are consumed as compiled assemblies whose types the host language cannot even name — the compiler refuses the program that tries, and the refusal was obtained by invoking the compiler on one written to fail (Appendix A: `RepertoireBoundaryLab.cs`). The registration gate admits a repertoire’s internal types (`DomainLibraries.cs:151`) and the actor reaches their members by reflection (`ActorHandler.cs:4457-4516`). The composition happens anyway, which separates two properties usually fused: *composition does not require host-language accessibility*. But reaching a type the host cannot name is a capability of the runtime, not a permission granted by the type’s author, and whether standing on the far side of that boundary amounts to an entitlement is a normative reading the compiler is silent on. This paper declares it as a reading. A reader may decline it and keep every measurement.

4.3 What the arrangement cannot supply

The honest boundary of the subject’s position, measured rather than conceded in the abstract.

Values that enter an act from outside the computation cross a **parameter plane**: captured once at the moment of the act, journaled with it, re-injected on replay rather than taken again. A clock reading is handled that way — on replay the recorded occurrence time is injected back as the act’s **Now** rather than the clock being read again (`ActorHandler.cs:1456`) — and so is an expression parameter, resolved once and frozen into the entry’s arguments (`Parameter.cs:439-455`).

The boundary itself is not this paper’s discovery, and saying so costs nothing. Capturing nondeterminism at a recording frontier is the classic problem of record-and-replay (LeBlanc and Mellor-Crummey 1987; O’Callahan et al. 2017), and durable-execution runtimes enforce exactly this discipline for workflows: an orchestration may not read the clock or mint a fresh identifier, replay-stable substitutes are supplied in their place, and the equivalence of the programming model with its record-replay implementation has been proven (Burckhardt et al. 2021). And beneath both, the discipline is the actor premise on the value axis: an actor’s next state is a function of the message it received and the state it held, so a value drawn from the machine behind the plane is an input with no sender — which is why Appendix C states the boundary as a condition on the host interpretation rather than as a rule of authorship. What this paper adds at the plane is only the measured failing case at its edge, met while composing repertoires the subject does not own — a value minted inside a foreign factory,

which no substitute could have intercepted because the assembler is never asked for it. The plane has an edge, and one value in the composed verb falls behind it: one repertoire mints its own identity *inside* a factory, so the value never crosses the plane, because the assembler is never asked for it (the call site is marked, by number rather than by intent, in `SubscriptionPaymentBridge.cs:38-72`; the measurement is `RetrievalIsALookupLab.cs`, which measures this and only this: the supplied key stable, the minted identity fresh, and retrieval a lookup in the subject's own state; it says nothing about capture). The repair the plane itself would offer is out of reach by the grammar of the case, and saying so exactly prevents a misreading: an expression parameter can draw a value once at the plane and freeze it into the entry, but only the repertoire could accept it, and this factory's signature asks for none — the identity is born inside the operation, as part of its own domain event. Making the factory ask would repair replay by editing the repertoire, and an edited repertoire exits the paper's category: the composition under measurement is of repertoires the subject does not own and did not touch (§5.2 counts the edits: zero), and the corpus entered under transformation rules fixed before measurement (Appendix A) — so the value was left as the repertoire produces it, and reported. Measured, live and replayed:

the assembler's key	stable across replay
the domain's identity	a different value on every replay

Replay re-runs the act, and the minted value returns fresh — an outcome the criterion's equivalence absorbs (§3.1), and which Appendix C proves is the identity replay can promise, exact equality being unavailable under its host axiom under its host axiom. It is the measured value that forces §3.1's clause to be declared before the criterion is used: without it, §3 would fail this paper's own entry. In the calculus of Appendix C the same discipline is the axiom on the host interpretation, and this value is its licensed minting measured live: fresh names differ per replay by construction, which is the bijection Proposition 4 proves is the strongest identity replay can achieve, and the clause §3.1 states. Two things would take down replay equivalence and nothing above it — a true violation of that axiom (uncaptured I/O, an escaped clock, foreign mutable state), and the *capture* of a minted identity into a later act's recorded arguments — and the definition and its exercises stand in the record under either: the verb is no less defined, exercised or owned. Nothing here is a defect of either party — the repertoire was written to be called once, by a caller that would store what came back. And what the measurement locates is not a replay failure, since the equivalence holds; it is an **authority boundary**. Durable-execution runtimes stabilize such values by supplying replay-stable substitutes from the runtime's own facilities; the domain's author could stabilize this one by asking for it. The assembler alone can do neither, and the cost lands on the subject: **it inherits the decision without acquiring any authority over it**. One line separates properties usually spoken of together:

A repertoire can be complete, executable and testable — and still
mint identities that no assembler has the authority to stabilize.

5. The Case That Makes the Plane Visible

5.1 The constitution may draw from anywhere the subject reaches

Nothing in §2–§4 required two domains. An assembled verb may be constituted from one operation, from several operations of one repertoire, from operations of several repertoires, or from a mix that includes statements only the subject’s own plane can contribute — its computation over what the repertoires said, and its speech:

Figure 1 — the forms a constitution may take.

<code>V := A</code>	
<code>V := A; B; C</code>	one repertoire
<code>V := A; X; Q</code>	several repertoires
<code>V := A; print x</code>	the subject’s speech, outward
<code>V := A; tell t x</code>	the subject’s speech, to another subject

What makes V assembled is not the plurality of its operations, nor of the repertoires they come from. It is that the resulting capability is authored at the assembly plane.

The constituents determine what is *available* to assembly. They do not determine the verbs assembly may author from it. And the caption says *authored* rather than *selected* because of the last two forms: assembly does not only compose repertoires — **the plane has vocabulary of its own**. *Print* and *tell* are the subject’s speech, outward and to another subject, and neither is writable inside a domain: an operation has no voice, no say over destinations, and no acquaintance with the subjects that future verbs will address. Both are earlier papers’ subjects and are not re-argued here. Read downward, the figure is the paper’s ladder in miniature:

domain operation	<code>A()</code>	what a repertoire can compute
assembled computation	<code>A(); B()</code>	what this subject can do
assembled speech	<code>print x</code>	what this subject can say
assembled address	<code>tell t x</code>	whom this subject can act toward

And the deduction must be kept honest here, because a repertoire could not write the *tell* even if its author wanted to: *tell* is a statement of the subject’s plane, not of the host language a domain is written in. What a repertoire’s author *can* write is the equivalent the corpus actually contains — publish an event, call a bus — and that equivalent has a measured shape (§6): it authors a transport, a contract and an implied destination into the piece, and it fixes in advance one of the future acts the piece may participate in, because its author must anticipate the subject it will someday serve:

inside a repertoire	at the assembly plane
<code>A() { ...; bus.Publish(ADone(x)); }</code>	<code>A remains A</code>
a transport, a contract, a destination	<code>V1 := A; print x</code>
authored into the piece	<code>V2 := A; tell t x</code>

A did not change; what changed is who is using A to say what. This is the separation of authorities the series keeps meeting, on a third face now: producing a value grants no authority over its destination; providing an operation grants no authorship over the verbs that will incorporate it — and imposes no duty to know whom those verbs will someday address. A repertoire need not know the sentence in which one of its operations will someday become a word.

One more form deserves its own exhibit, because it may be the purest of all:

```
W := balance = a.Balance();    a query verb: reads across repertoires,
    points = x.Points();       relates what no one of them holds,
    print balance + points      and says it
```

A query verb such as *the customer's position* — a balance from one repertoire, points from another, pending orders from a third, related and pronounced — mutates nothing, may persist nothing, and is still undeniably a capability of the subject: it says something no piece can say, because it occupies the position that sees them jointly. The width of §4.2 is demonstrated there in the computation itself, with no boundary machinery in sight.

One discipline keeps that exhibit from licensing what the paper has already ruled out, and the sketch is written to obey it. The computation admitted at the assembly plane is **the relation itself**: arithmetic over what the repertoires said, of the kind no repertoire could host — a formula whose subject matter spans two repertoires cannot live inside one without lodging exactly the cross-repertoire knowledge §5.2 measures as absent. A formula whose subject matter lies **within** one repertoire is that repertoire's operation, and assembly *invokes* it rather than authoring it. This is not a style rule laid on top of §4.2; it is §4.2 applied to expressions: authoring single-repertoire logic at the assembly plane would be depth, and the assembler's reach has none.

The same honesty is owed to the sketch's humblest statements. Assignment is vocabulary the plane shares with every language, and the paper claims none of it. What no repertoire could write is the **scope** those assignments fill: **balance** and **points** bound side by side in one place, a pair neither piece could hold — not because binding is special, but because those names cannot co-occur inside either (§5.2). The same scope holds values whose types the host language cannot even declare a variable of (§4.2). The subject's typed scope is the position that sees the repertoires jointly, made operational; assignment is merely how a constitution moves values among its operations — the third of §2.2's constituents of unity, at work.

The plane — verbs of the subject, above operations of the repertoires — is the same in all of them. But one form makes the plane *visible* in a way the others cannot, because it removes the reading that V is shorthand for something one repertoire could have said: when the operations come from repertoires that share nothing, no piece's vocabulary can even in principle be the verb's home.

5.2 One story over two repertoires that share no type

An author-constructed scenario — written for this study, and evidence of realizability only (Appendix A: `ALongTrajectoryOverTwoUntouchedRepertoiresLab.cs`). A customer buys a subscription, which is paid and granted; then places an order, adds items, has a payment method verified; the order is paid and shipped.

domain operations invoked	17
aggregates taking part	4
shipped assemblies, different authors, unmodified	2
types the two assemblies share	0
test doubles	0

It runs, end to end, with no handler, no bus, no repository, no mapper — the claim being narrower than their dispensability to the systems that use them: none is required to express *this trajectory* in the repertoires’ own operations.

And the run settles §1.1’s corollary by construction rather than by argument. The two assemblies were authored years apart, by parties who never met, and nothing in either was written toward this verb — yet seventeen of their operations constitute it. **A repertoire need not anticipate the verbs in which its operations will later participate** is not a hope here; it is the condition the scenario ran under — §1.1’s deduction, run as an experiment.

And there are things true of the trajectory that no participant in it can state. The customer appears as a Guid-bearing identity in one repertoire and a string-bearing one in the other; neither assembly names the other; so neither repertoire is in a position to state a proposition whose subject matter spans both — *this customer bought a subscription and later bought these items* has no possible position inside the run from which it can be said. For either repertoire to assert it, the relation between the two representations would have to become knowledge held inside one of them, and the measured zeros show neither published repertoire occupies that position.

The whole is knowable, but not by any of its parts.

The run raises nine domain events — the corpus domains’ own, emitted by their aggregates, in a scenario this paper composed, so the count carries the constructed kind of §1.2’s table and no weight about either corpus system’s design — every one naming a transition of a single aggregate, none naming the trajectory:

The repertoires name every transition they are entitled to name. What they do not name is the trajectory that exists only in their composition.

Under the arrangement of §2 that same trajectory is one entry — the entry printed there — and the subject whose verb it is can be asked for it again.

6. Returning to the Corpus: Where Is V, and Where Did the Continuity Go?

The order of the argument matters here, and it is the reverse of the archaeological one. This paper did not infer a missing category from the corpus and then build something to fill it. The category was demonstrated positively first — §2 constituted V, exhibited its one definition, exercised it twice, and replayed both exercises — and the corpus now serves as a **control**: does it contain a capability at V’s level?

That inversion changes what the corpus evidence has to carry. It no longer has to prove an absence by exhaustion; it has to answer a yes-or-no question about a category whose existence is already established. The question is one word: **where is V?** And behind it, §1.1’s question in the past tense: **where did the continuity go?** The trajectory runs — the corpus works — so if no unit holds its continuity, the continuity went somewhere, and this section finds it.

What is found is A, B and C — and their events, their handlers, their correlations, their coordinators’ state. What is not found, anywhere, is V as a capability of a subject.

The census. Every named artifact of both subsystems’ vocabularies was enumerated and coded, and both steps run from published rules rather than from a reading (Appendix A: `EveryNameStaysWithinOneSubjectsVocabularyLab.cs`, whose header is the codebook and whose body is a coder with no judgment to exercise). The population rule reproduces the denominator mechanically from the pinned corpus: 27 domain events, 16 integration events — the subsystem’s declared events plus the foreign events it handles — and 29 commands, with handlers, generic infrastructure wrappers and queries out of population: **72 artifacts** across three layers and two grammatical moods. The construct — *names a trajectory spanning subjects* — is coded by two rules, because one alone is blind in exactly the form that matters. Rule one is mechanical, and generous within its form: an artifact is coded as naming a trajectory if its identifier references two or more distinct aggregates of its own system — the lexicon derived mechanically from the corpus — with no verb, no ordering, and no judgment required. But a trajectory can be named without naming any constituent: *CompletePurchase*, this paper’s own opening example, carries no aggregate noun, and a rule that counts aggregate references cannot see it. So rule two hands to **manual coding**, row by row and each with its reason published, exactly the identifiers rule one cannot see: every identifier referencing no aggregate at all, and every identifier bearing a word from a published process lexicon (*checkout*, *onboarding*, *fulfillment*, *purchase* and their kin) — and a residual identifier without a published coding fails the run, since silence is not a coding. Coded under both rules the count is **zero across 72**: mechanically zero under rule one, and a

three-artifact residual under rule two, each naming a single subject’s transition, each reported with its reason. What this is not has to be said as plainly as what it is: **no independent coder was employed**. The two codings that agree — the author’s reading, and rules the author wrote after that reading — are two instruments of one authorship, so their 72/72 is internal consistency made auditable, not inter-coder reliability; and with every artifact in one category there would be no variance for a chance-corrected coefficient to correct anyway (Cohen 1960). Re-running the lab verifies that the published rules produce these codings from the pinned corpus — verification, not a further coding, since a deterministic rule is one coder however many times it runs. A further coder is a reader who codes the 72 against the construct and compares, which the codebook and the coded table exist to make cheap. Every artifact stays within the vocabulary of a single subject. The absence cannot be explained by a naming convention: it persists in the two layers where naming was unconstrained. The clearest single case is a pair of commands, one asking for a subscription’s payment, the other for the subscription that payment entitles — two imperatives for the two halves of one act, and no imperative for the pair. And after §2.2 the census means something sharper than it could have meant as an opening exhibit: **the missing name was evidence, not the thing missing**. The constituted verb of §2 has no name either. What the corpus lacks at the level of the trajectory is not a word — it is the unit the word would have named.

Defeat condition, cheap to run: exhibit one artifact in either system — event, command, type or method — whose name refers to a trajectory spanning operations of more than one subject. The count is zero across 72; one takes it to one, and either form is accepted: two subjects’ nouns in one name, which rule one takes mechanically, or a single word naming the whole — a CompletePurchase, a Checkout — which rule two’s manual lane exists to see, and whose lexicon any reader may extend.

The dissection. Six rules, pre-registered and committed before any removal, were applied to both systems’ write surfaces in audited two-commit steps, with the hole count read from compiler diagnostics; the full procedure, its amendments, its stopping rule and its unequal-objectivity declaration are in the artifact. Two results matter here. The authors’ own domain suites — 49 tests — pass unedited against the dissected assemblies, so **the operations already computed on their own; what the dissection had to free was the build**. And what remains of a write step after persistence, ambient context, transport, wiring and observability are removed is: invocations of domain operations, plus a residue that articulates them. The pieces of V are everywhere. V is nowhere.

One reading of the persistence rule follows from §2.4 and is offered as a reading: removing persistence from a domain does not merely make it portable — it returns a decision to a plane that has context for it. A domain with persistence inside it has one author deciding both what the domain means and what survives as history; a domain without it leaves the second decision free to be taken per performance, by the party constituting the verb. Nothing in that is a judgement

of either corpus system, whose authors took both decisions deliberately and well.

The compensations — where the continuity went. Where the plane has no author, the software compensates, and the compensations are measurable. An arrow between two halves of one act is carried by a durable enqueue into the module’s own schema, read by a poller in the same process — transport machinery supplying only *and then*, since the same act composed directly, through an in-memory queue, or through a keyed register delivers identical values, differing only in the identity a repertoire mints for itself (Appendix A: `WhatTheCarrierCarriesLab.cs`). Events that carry a whole aggregate are consumed by handlers that read an identity off them and fetch the aggregate again. Each mechanism buys real guarantees its system needs, and they are what movement looks like when the composition has nowhere to live as a unit — **continuity authored into the pieces**, because no other place could hold it. What §2 changes is not that continuity exists but where it can live: once the whole is a verb of a subject, the pieces no longer have to carry it.

7. The Neighbours

A claim this visible owes an answer to an uncomfortable question: if assembly producing a capability were real, how could five decades of languages, patterns, planners and runtimes not have produced it already? The honest answer is that most of V’s ingredients were produced, some long ago, and this section says so before saying what is left. The search behind it was run to find the paper that would make this one unnecessary, and the neighbours below are what survived it. The search is itself reported as method in Appendix B — its query strings, its sources, and the adversarial-review channel whose seven contributions measure what the strings missed; the claim it supports is scoped there: absence over the examined set, with §3’s questions as a finite test for any candidate beyond it.

One objection organises them all:

“You have merely made composition first-class.”

Every neighbour is put the same single question — **what exactly is V in this model?** — unfolded into the battery the paper has already answered for itself:

does V exist separately from A, B and C?

can V be reused, with parameters?

are V and V(p) distinct?

whose capability is V?

who assigns each exercise’s command/query modality?

who decides whether V(p) belongs to the subject’s history?

The first three are old, and the neighbours below hold them in various combinations. The last two are where this paper’s difference proves hardest to reduce, and no examined neighbour holds either (Appendix B).

7.1 The composition ancestors — conceded first

Procedural abstraction is the oldest and deepest. A function is a definition exercised many times; that structure is the founding gift of every programming language, and §2.3’s three levels reproduce it deliberately. What a function definition lacks is not reuse — it is everything below the third question: it lives in program text, belongs to a namespace rather than a subject, and §3.3 measured what remains of it after a run.

The Command pattern and its composites (Gamma et al. 1994) make the *request* a first-class object; a MacroCommand executes a sequence of Commands, and macros are built from existing commands. The pattern even keeps a history — the invoker’s undo stack is a record of invocations. Each piece of the battery’s first half is here. What is not: the macro is program structure, not an entry in anyone’s history; the undo stack is the invoker’s ephemeral memory, not the subject’s record; and the modality is intrinsic — a Command *is* an execute, not a verb whose historicity the assembler attributes per constitution.

Transaction Script and the application service (Fowler 2002) organise domain operations to carry out a use case, and the corpus of §6 uses them well. The body can be identical to V’s, which is why §2.5 refuses to answer this neighbour from the body. §3.3 answers it from what survives execution.

Workflow systems (van der Aalst and van Hee 2002) already hold the definition/instance separation at the process level: a workflow definition is instantiated as cases, and subworkflows make pieces of a definition reusable across it. So *defined once, exercised many times* cannot be claimed even at the granularity of business processes. What a workflow case’s audit trail is, though, is the engine’s record of an enactment — everything is logged because the engine logs, not because a party attributed historicity to this verb and not that one; and the definition belongs to the process model, not to a subject with a history of its own.

Algebraic effects and handlers (Bauer and Pretnar 2015; their language Eff) make computational effects first-class, definable and composable — one more form of *composing operations yields a new abstraction*, from the programming-languages side, and conceded with the rest.

Multi-stage programming (Taha and Sheard 1997; MetaML and its descendants) builds, at run time, a program value that is then run with different arguments — the battery’s first three questions answered from the same side, and more literally than by any neighbour above: the composed definition exists at run time, is distinct from its executions, and parametrizes. What the staged program is not is anyone’s capability. It is a value in the host program’s hands; its persistence is the host’s own concern; no record holds its definition or its exercises; and no modality attaches to either. Stage separation answers *when* code exists — the plane here answers *whose record it enters*.

Macro-operators are the sharpest ancestor, and the concession here has a

date: Fikes, Hart and Nilsson (1972) generalise a STRIPS plan so that problem-specific constants become parameters, store it as a triangle table, and reuse it — in whole or in part — as a single macro action in later problems. *One definition, many exercises, with different bindings* is fifty years old, and this paper claims no part of it. **Options** (Sutton, Precup and Singh 1999) do the modern form: temporally extended actions added to an agent’s set, used interchangeably with primitives. Both put a composed capability into a subject’s repertoire.

What the battery still separates, and it is the same two questions each time: the MACROP lives in a plan library and the option in a policy set — neither the definition nor its exercises enters the subject’s *history*, and nothing attributes modality or historicity per exercise, because in a planner there is nothing for that distinction to govern. The constituents, moreover, are the agent’s own operators; none of these ancestors composes repertoires their subject does not own and cannot name (§5.2).

Definitions emitted at first execution have an ancestry of their own, and §2.2’s runtime-emitted definition belongs to it. Programming by demonstration records a reusable macro from an enacted sequence (Cypher 1993); a trace-based JIT emits a reusable compiled trace the first time a path runs hot (Bala, Duesterwald and Banerjia 2000); and procedural reflection made the running program an object within its own system (Smith 1984) — a tradition made operative by the metaobject protocol (Maes 1987; Kiczales, des Rivières and Bobrow 1991), in which a running system extends its own executable vocabulary. All conceded — with the by-now familiar difference of address: what those systems emit or extend lands in a macro library, a code cache, or the image’s own machinery — an artifact of the program, not an entry in the emitting subject’s record.

7.2 Durable execution

The nearest living neighbour is not a pattern but a shipped runtime family, and the first round of discovery strings missed it; the adversarial-review channel of Appendix B supplied it, and the index pass then reached it on its own under revised strings — the miss and its bound are both recorded there. In **Durable Functions** (Burckhardt et al. 2021), an orchestration is a parametrized definition invocable many times with different arguments; the activities it composes come from libraries the orchestration does not own; *entities* supply actors with state and a journaled history; the runtime records invocations and re-executes by replay — and the paper proves the equivalence of the high-level semantics with the record-replay implementation. Orleans is the actor ancestry (Bykov et al. 2011). Temporal and Cadence carry the same model as industrial practice without a refereed venue; that Temporal’s replay has the same shape measured below — the workflow’s code re-executed, its activities run once with results cached, determinism enforced by rule — is documented from the actor community’s own side (Bernhardt 2021), which extends the engine measurement to that sibling documentarily.

Put the battery to it without flinching. The first three questions are satisfied outright. So is much of §5: the composed pieces need not be the orchestration’s own. And the determinism frontier this paper met at §4.3 is there *formalized and enforced* — the prohibition on reading the clock or minting an identifier inside an orchestration, with replay-stable substitutes supplied, is the parameter plane as a language rule. Nothing about that frontier is this paper’s, and §4.3 now says so.

What the criterion of §3 still separates is stated here from the published semantics rather than measured, and is flagged as such. In durable execution **the definition is code**: the orchestration function lives in the program, and the history stores the events and outcomes of its activities; replay re-executes the code against the recorded history to reconstruct position. Ask §3’s question (a) of that history and the answer is invocations and results — closer to constitution than any neighbour above — but the constitution itself never enters the record: change the code, and the same history replays against a different definition, which is precisely the versioning hazard that model’s own documentation manages. In the arrangement of §2 the definition *is* an entry; the constitution is data in the subject’s history, and §2.3’s second use referenced it there. The hazard is thereby narrowed, not escaped: the repertoires’ implementations stay ambient in both models, and a changed operation replays the same entry differently — what moves into the record is the composition, and §8 carries the remainder as a limit. The second separation stands whole: durable execution attributes no command/query modality to any exercise — recording is how execution works, not a decision anyone takes per performance, so *which performances count as the subject’s history* is not a question the model offers to anyone. And the first separation is not documentary either: Appendix C’s Proposition 5 derives it from the shape of the arrangements — its negative half needs neither the host axiom nor anything about this engine or that one, only that an ambient program is free to vary — so what changes with a runtime’s documentation is the illustration, not the result. Both separations are now more than documentary. §3’s coordination arm was run on this family’s own engine — the Durable Task Framework under its in-memory emulator, with the history read through its public dispatch middleware — and the measurements landed where the semantics say: the record holds all six invocations, and replay re-executed the orchestrator’s code in six replay episodes while re-performing no operation (Appendix A: `TheCoordinationArmOnAThirdPartyEngineLab.cs`). Its own authors describe the model in exactly these terms — “an abstraction of a persistent call stack”, every step “transparently persisted by the server”, and “a workflow is a durable actor with all its state fully preserved” (Bykov and Fateev 2021) — a stack persisted, a definition that stays in the program. The versioning hazard remains cited from their semantics rather than measured.

7.3 Behaviour acquired by an object

One neighbour is closer than any pattern, and it was surfaced by the adversarial-review channel of Appendix B rather than by the paper’s own genealogy. **Talents** (Ressia, Gırba, Nierstrasz, Perin and Renggli 2014) are dynamically composable units of reuse applied to *an individual object*: a talent specifies methods that can be added to — and removed from — the behaviour of one object, without touching the other instances of its class. That is dangerously near *a subject acquires a repertoire of its own*, and the paper cannot brush past it.

The answer is in what each thing adds and where. A talent adds **methods to the object** — the object’s own behaviour grows, and the new methods live where its old ones do. Assembly adds **nothing to any object**. The repertoires’ types are unchanged — §5.2’s are shipped binaries — and V is constituted on another plane, out of operations that remain exactly where and what they were. What grows is not an object’s method set but a subject’s repertoire, and the subject is not one of the objects. From there the last two questions of the battery finish it: a talent’s methods have whatever modality methods have, and no notion of which of their invocations count as anyone’s history.

Prototype languages put the same acquisition in the object system itself — in Self (Ungar and Smith 1987) an object holds behaviour without a class, and can acquire it — and the disposition is Talents’: the repertoire that grows is the object’s own, the growth lives in the running image, and no record of the definition or its exercises survives into any subject’s history.

7.4 Command sourcing

Recording invocations and replaying them through their handlers is an established alternative to recording outcomes, and it is the neighbour closest to §2.1’s mechanical property. Two things separate it. Each recorded command is self-contained — there is no derived definition that later entries reference, so the record holds exercises of nothing. And the known objection to replaying commands — re-execution meets nondeterminism — is precisely the boundary the parameter plane draws (§4.3): what crossed the plane replays; what was minted behind it does not, and this paper reports its own failing case there.

7.5 The positive control: the materials without the category

No new neighbour is needed for this one; two already present supply it jointly. Command sourcing (§7.4) and the process manager (§7.6) live in the ordinary event-sourced stack: commands received, events appended to a journal, state rebuilt by replay, reactions and coordinators reading the log. Every material of §2 is present in that stack, at industrial maturity — and the unit its literature is about remains the aggregate, the handler, the process. The question its records answer stays *given this state and this event, what follows?*, not *which verb did this subject acquire?*

That is not a shortcoming to be argued with; it is a control this paper needs. Having the materials does not by itself produce the category — a system can hold commands, logs, events and reactions and never constitute a verb of a subject. Which is exactly why the category had to be measured (§2–§3) rather than asserted, and why this paper never claims that the pieces were unseen.

7.6 The process manager

One arrangement deserves its own subsection because it is the one this paper is most easily mistaken for competing with, and it is conceded entirely.

A process manager (Hohpe and Woolf 2003; its transactional ancestor is the saga, Garcia-Molina and Salem 1987) relates already constituted acts into a trajectory: correlation, lifecycle, resumption, compensation. It is mature and well understood, and nothing here improves on it, competes with it, or is offered as an alternative to it. The two relations are orthogonal, and the difference is not temporal — an assembled verb may contain operations separated by any amount of time; verticality is not simultaneity:

The horizontal relation presupposes its acts. The vertical relation constitutes one.

The same community also runs the arrow in reverse, and the reverse deserves naming. **Process mining** (van der Aalst 2016) discovers a process definition from event logs — from exercises to a definition, the inverse of §2.2, and precisely the archaeological direction §6 declined for itself. What discovery yields is an analyst’s model *about* the subjects whose logs were mined: it lands outside every subject’s record, no subject can exercise it by reference, running it executes a model rather than re-performing anyone’s acts, and it attributes nothing — its input is whatever the systems happened to record, which is the attribution this paper locates at assembly. The kinship is exact and inverted: mining reconstructs, with effort and approximation, a definition the record never contained; constitution writes the definition into the record so that nothing ever needs to reconstruct it.

Because the axes are independent, the two appear together naturally: five operations may constitute *Purchase* vertically while *Purchase*, *Fulfilment* and *Delivery* form a coordinated trajectory horizontally. **An assembled verb may itself be one of the acts a process manager later relates; constituting an act does not remove the need to coordinate acts.**

A process manager answers what follows this act. An assembled verb answers what belongs to this act.

The observable separation between the two is §3’s, and it is about what one record contains — not about identity, streams, state or names, all of which a process manager legitimately has.

7.7 The older neighbourhood

That some acts are *constituted* rather than merely caused — that a movement of pieces can **count as** an act of another subject — is not a question software raised first. It is formalised in deontic logic and in the study of normative multi-agent systems, where *counts-as* is a term of art (Jones and Sergot 1996), with the constitutive-rules lineage running through Austin (1962) and Searle (1969, 1995); none of that is claimed as new here. From the process-modelling side, business-artifact approaches centre a process on an entity with a lifecycle and a history of its own (Nigam and Caswell 2003) — subject-shaped modelling, without the constitution or attribution this paper measures. Enterprise ontology reaches the adjacent ground from organisational modelling: in DEMO (Dietz 2006), an organisation is a network of actors and transactions, actor roles are the units of authority and responsibility, and authority is assigned on the basis of competence — a normative account of who may perform what, stated for organisations of people. This paper’s §4.2 is the mechanical shadow of that question. What this paper adds is not the question but a mechanical residue of it in an ordinary artifact: the boundary of a composing party’s reach is measurable in a build, with the normative half explicitly left open (§4.2), and whether a composition was constituted or merely coordinated is decidable from a record (§3). The literature supplies the concepts; the arrangement supplies something the concepts can be checked against.

7.8 The neighbours in one table

Seventeen rows: the sixteen neighbour families of Appendix B, plus the event-sourced stack, which is §7.5’s positive control rather than a family the search found.

neighbour	already does	the question this paper adds
procedural abstraction	definition exercised many times, in program text	what remains of the composition after a run?
Command / MacroCommand	the request as object; macros from commands; an undo history	whose capability, and who attributes its modality?
workflow / subworkflow	a definition instantiated as cases	whose history is the audit trail, and who chose what enters it?
algebraic effects	composed operations as a new first-class abstraction	a subject, a record, a modality — none is in the construction
multi-stage programming	a program builds a program at run time, reused with arguments	a value in the host’s hands: whose capability, whose record?

neighbour	already does	the question this paper adds
Talents	methods added dynamically to one object	V adds no method to any object; the repertoire that grows is another subject's
reflection / MOP / prototypes	a running system extends its own executable vocabulary	the growth lands in the image's machinery, not in any subject's record
transaction script / application service	A; B; C as a use case	did a capability of a subject appear?
macro-operators, options	a parametrized composition enters the agent's repertoire	do the definition and its exercises enter the subject's <i>history</i> ?
macro-by-demonstration, trace JIT	a definition emitted at first execution	emitted into whose record?
durable execution	parametrized orchestrations, replayed from a journaled history, determinism enforced	is the <i>constitution</i> in the record, and who attributes modality?
command sourcing	invocations recorded and replayed	exercises of <i>what?</i> and what replays deterministically?
access audit logging	reads journaled because the disclosure must stand	attributed per performance, at assembly, into the subject's own history? (§2.4)
the event-sourced stack	commands, events, logs, reactions — every material of §2	the control: materials alone do not produce the category
process manager	relates acts into a trajectory	which operations constitute one act?
process mining	recovers a process definition from event logs	the definition lands in an analyst's model — the inverse of writing it into the record
counts-as, DEMO	constitution and authority as concepts	a build and a record the concepts can be checked against

The neighbours attack different rungs, and none needs to be defeated — each

is conceded at its own task. Composition has been made first-class, definitions reusable, behaviour dynamically acquirable, coordination explicit and constitution formal — separately, and in combinations that come close. What this search did not find said together is the conjunction — stated as a finding of the search reported in Appendix B, its adversarial-review channel included, scoped to its examined set, rather than as a proof of novelty:

Assembly is a plane at which operations available from repertoires become verbs of another subject; V is distinct from its exercises; and each exercise’s command/query character — and therefore which performances belong to that subject’s history — is attributed at that plane rather than inherited from the constituent operations.

The pieces of this paper are not new. The claim is about the planes they usually collapse: that *what can be computed*, *what the subject can do*, *what the subject does* and *what counts as its history* are not the same decision — which is the register this series has used from the start, discovering that things habitually fused are separately decidable.

8. Limits, and What Is Not Claimed

The cost measured is the record’s; no advantage is claimed. Three prices are measured against the direct baseline the criterion already uses (Appendix A: `WhatTheRecordCostsLab.cs`, the run published as `data/paper0A-assembled-verb/price-release.log`), every timed quantity over ten runs and reported as its median with the min–max spread, on a declared environment: Release build of engine and harness, .NET 9 on Windows 11, one 32-thread desktop processor, workstation GC, wall clock, the corpus assemblies prebuilt binaries in both arms — and the direct baseline performs the same three statements and allocates the same domain objects, so what it lacks is only the record. *Growth* (exact counts, not samples): the act carrying the definition and first exercise costs 554 bytes of journal; each additional exercise costs ~112 — reuse is visibly cheaper than restating, which is the capability claim’s material side. *Replay*: reconstructing the subject from a record of 100 exercises takes 4.6 ms [4.4–5.6]; from 1,000, 15.0 ms [13.9–20.2]. *Throughput*: the plane performs ~2,490 exercises per second live [2,310–2,562], where the same statements as direct in-memory calls run at ~529,000 loops per second [476,000–542,000] — a ~**210x** price, the ratio of the two medians, reported to two significant figures because that is what the instrument supports: repeated invocations of the lab, each with its own warm-up and ten samples per arm, move the multiplier by a few percent. Two sampling decisions are part of the measurement rather than incidental to it. A warm-up run is discarded in every arm, so that JIT compilation is not sampled as steady-state work. And a direct *sample* is ten loops of the same thousand, each over a fresh register, timed together and divided back: one loop of a thousand lasts about two milliseconds — the scheduler’s noise floor rather than a measurement — while scaling a single

loop to ten thousand would have changed the work itself, since the register would grow ten times larger. The comparison is asymmetric by construction and the asymmetry is the finding: the direct loop buys execution only; the plane’s loop buys execution plus a definition exercised by reference, an attributed modality, and a record that reconstructs the subject. The decomposition the two medians support: replay re-performs the same thousand exercises in ~15 ms — about eight times the direct loop’s median, against the plane’s ~210 — which locates the dominant term outside the plane’s interpretation, in the durable write — an inference from two measured quantities rather than a third measurement. What remains unmeasured: concurrency, scale beyond one machine, and every competing arrangement’s own price — whether this arrangement is cheaper or faster *than any other* is unexamined.

Two implementations, one author. The plane’s realizability no longer rests on a single engine: a prototype of a few hundred lines implements it on a third-party engine, the same engine that carries a coordination row, so the substrate is controlled (§3.2). What remains true is that both implementations are this paper’s. An implementation by an independent party, put to §3’s criterion, is the measurement this paper cannot supply itself.

The corpus is two systems, selected for inspectability and reputation, not representativeness. The census was run after the selection, and its zero is a statement about these two systems: *the plane did not appear of its own accord in either*. Whether it fails to appear elsewhere is not asserted, in the abstract or anywhere — no population-level claim is made. And the census’s coding employed no independent coder: the codings that agree are the author’s reading and rules of the author’s writing (§6). The published codebook makes an independent recoding cheap, and none has yet been performed.

Ownership is measured; subjecthood is assumed. The paper establishes ownership relative to the actor whose record contains the definition and its exercises: V belongs to no constituent repertoire (§5), its exercises are attributed to one actor, and replay reconstructs that actor from its own record. It does not derive subjecthood from constitution — and Appendix C says the same of itself: its Op/Verb partition takes the subject as given, so the calculus carries the separations *given* a subject and derives no subject from them. That the owner is a subject — the frame in which its record is a history and its outputs are speech — is inherited from the actor model (Hewitt, Bishop and Steiger 1973) and from this series’ prior finding that identity precedes staging. A reader who declines the frame keeps every measurement and loses only the names.

The entitlement reading remains a reading. §4.2 concedes that reaching past host-language accessibility is a runtime capability, not an author’s permission. A reader who declines to call the subject’s position a *right* loses nothing measured in this paper.

One value in the composed verb returns fresh on every replay, and no one at the plane has the authority to stabilize it. An identity minted

inside a repertoire’s factory changes on every replay; the criterion’s equivalence absorbs the freshness, and Appendix C proves that exact identity is unavailable under its host axiom while that equivalence is — so what stands is not a replay failure but an **authority boundary**, reported against the paper’s own flagship entry: the assembler can neither supply the value nor stabilize it, where durable-execution runtimes stabilize theirs by design and a domain’s author could by asking (§4.3). The calculus also states the condition under which the guarantee does hold — every reference to a minted value travelling between acts as a lookup under a key that crossed the plane — which is a characterization of when replay is safe, not advice about how to write verbs (Appendix C). What the arrangement supplies is otherwise detection rather than immunity: the corpus’s own suites never meet this value — its 49 tests pass over it — and replay is the instrument that makes the undeclared inbound visible, against the actor premise, a standard the repertoire never adopted, which is why §4.3 charges neither party. The value is also the measured instance that forces §3’s outcome equivalence to be declared before use (§3.1). The subject inherits a decision it cannot author around. Completeness, executability and testability are the repertoire’s properties; a replay-stable minted identity is not among them, and no assembler can add it from outside — this corpus separates the four.

A captured mint breaks replay equivalence outright, and this one the calculus found first. Past the licensed freshness sits a sharper hazard: a minted identity read by the assembler and *captured* into a later act’s recorded arguments. The recorded literal names a value the live run held and the replayed run never minted, so no renaming relates the two (Appendix C, Proposition 4’s boundary). Measured, the composition’s own invariant — the register’s key is the identity of the payment kept under it — holds live and fails after replay; the same composition referring to the value by a supplied key holds through both, its mint just as fresh (Appendix A: `WhereReplayEquivalenceBreaksLab.cs`). This is the paper’s one finding obtained by reasoning about the arrangement rather than by measuring what it was built to do: the derivation is committed before the lab that confirms it exists, and both commits are cited in Appendix A, so the order is checkable rather than asserted. Durable-execution practice prevents the case by design, supplying replay-stable identifiers from the runtime; this arrangement leaves it to what the assembler records, and the calculus states the sufficient condition under which the guarantee holds.

The record closes over the composition, not the operations. Replay re-performs by reading the repertoires’ code, which lives outside the record: change an operation’s implementation, and the same record replays to a different state. The criterion holds the engine and the repertoires fixed in (b) for exactly this reason, and Appendix C proves the relativized form: replay is a function of the record *and the interpretation* in every arrangement measured, and what distinguishes constitution from the strongest coordination is the remainder — an ambient program carrying the composition there, nothing but the operations here (Proposition 5). The versioning hazard §7.2 cites against durable execution is therefore inherited at the repertoire level, not eliminated. A subject that

needed its operations’ semantics to survive their authors’ edits would need more than this arrangement supplies.

The query modality is clean about the record, not about state. A query leaves no entry, and that is what §2.4 measures. What follows for state is narrower than it may read: the engine prevents a query from persisting, and in an arrangement where durable state *is* reconstruction from the record, what a query touched does not survive rehydration. Repertoire state held outside that record — statics, caches, external stores — is reached by a query like any other act, and returns from nowhere. Appendix C carries the same boundary as its second idealization.

Reuse is measured for one modality. §2.3 shows the verb drawn on again, and shows it for the journaled command, where exercises persist and can be counted. Query exercises may leave no record, so the same measurement cannot be run on them; their standing as exercises of the same capability is argued from the definition plane (§2.4), not measured on a record. The paper keeps the two apart.

Specialization is not measured at all. Whether assembled verbs can be *inherited* or *specialized* — whether a subject’s repertoire admits the structure that types give operations — is future work, and nothing in this paper asserts it. What is asserted is only the precondition such work would need: that the verb exists as a unit at all.

Recognition is a corollary, not a contribution. A reaction can bind values from two constituent operations of the composed verb precisely because both are in one recorded entry — the correlation is carried by the act itself. The mechanism runs on both planes of §2.3 at once: the pattern executes against the verb’s cached definition while the captures read the exercise’s journaled arguments (`Reaction.cs:1770-1795`), and entries whose composition was consumed at execution are explicitly skipped, having nothing stable to hold (`Reaction.cs:1825-1843`; Appendix A: `TheTrajectoryBecomesObservableLab.cs`). That is reported as a consequence: once something exists as a unit in the history, the rest of the system can refer to that same unit without reconstituting it. Reading those units — what recognition is, and what it entitles — is the next paper’s subject, not this one’s.

9. Conclusion

The paper opened with two arrangements side by side and one question: what exists in the second that did not exist in the first. The answer was earned in an order, and each step of it declares its kind: a measurement where the claim is empirical, a construction or a proof where the distinction is definitional — the division §3.5 states and Appendix C carries — and a declared reading where it is neither.

A capability was **produced**, in the operational sense §2.3 fixes and flags: one

definition, two exercises with different arguments, both replayed from the one record. Its recorded constitution is **distinguishable** from the arrangements it resembles, by a criterion any record can be asked — an event-sourced coordinator’s record contains none of the constituting statements, durable execution’s contains all of them yet re-performs none on replay, a trace contains all with no replay; only the constituted verb both contains and re-performs — an arrangement realized twice, once on this paper’s engine and once, in a few hundred lines, on a third-party one (§3.2). It is **anonymous**, an integer, while the method and the trace carry the business names — so naming was never the property. Its **modality is attributed, per performance**: one body, one verb, left no history exercised as a query and was journaled, replayable, performed as a command, the domain unchanged either way. Its constitution **drew on repertoires that share no type** and were never edited. And the corpus, asked *where is V?*, answered with 72 named artifacts none of which names a trajectory spanning subjects — coded twice, by the author and by a published mechanical rule, in full agreement — and 49 of its own domain tests passing over the freed pieces — the ingredients everywhere, the capability nowhere.

What survives the limits of §8 is therefore not a proposal but a conjunction, found standing among the examined neighbours of a search built to knock it down (Appendix B): a capability of a subject, constituted from repertoires the subject does not own, whose definition and exercises survive into the subject’s record, and whose modality — including which performances count as the subject’s history — is attributed at the plane of assembly.

As this series has done before, the paper closes by handing the reader its instrument rather than its enthusiasm. Of any system, take the record that claims to be the whole and ask: **how many of the statements that constitute the whole does it contain, and does replaying it alone bring the outcome about again?** And then the two questions no neighbour of §7 was found to hold: **who assigns each exercise’s command/query modality, and who decides which performances enter the subject’s history?** Systems will answer differently, and the answers are informative wherever they land; nothing obliges a system to have V, and §8 said so.

And every item just listed survives the flatter description §4.1 admits, for a reader who declines this paper’s vocabulary: the criterion, the ambient hierarchy, the attribution of historicity and the census are findings about records and engines, not about the words for them. The declarations of §8 fence that field; they do not empty it.

Two things are deliberately left standing. Whether occupying the assembly plane is an entitlement rather than a capability remains the open reading of §4.2. And once wholes exist as units in a subject’s history, what *reading* them amounts to — what recognition is, and what it entitles — is not begun here. A subject that can acquire verbs has a history worth reading, and that is the next question, not this paper’s.

The last consequence is the one the title was carrying all along, in the word *authorship*. The repertoire need not know the verbs in which it will participate. The subject need not have authored the operations from which its verbs are made. Between those two freedoms sits one operation:

Assembly turns the use of capabilities into a capability of a subject.

That is the finding. The consequence the series takes forward is the line after it, and it is the one §1.1 promised and §6 measured from the other side: the assembled verb matters less because it adds one capability than because it gives continuity somewhere to live other than in the capabilities it joins. The dissection showed the pieces already computing on their own; the carrier measurement showed the wires between them delivering the same values three ways — supplying *and then*, not meaning; and the long scenario showed two repertoires that never met joined in one verb, unmodified. The pieces kept their domain meaning without the machinery that carried their continuity. Give that continuity a place of its own, and:

Once the composition has a place of its own, continuity no longer has to be authored into its constituents — and the constituents need not know the future wholes in which they will participate.

Disclosure

Large language models were used in the preparation of this work: as the adversarial-review channel reported in Appendix B, and as drafting and verification assistance under the author’s direction. Nothing entered the paper on a model’s assertion alone — every contributed source was verified against the primary record, every count resolves to the labs or the dissection, and the model identities and versions are recorded at deposit. Venue-specific disclosure requirements are checked at each deposit.

Appendix A — The Labs

Every count in the body resolves to one of two published artifacts, and the split is stated so it can be checked. The measured counts — the criterion’s cells, the reuse and modality counts, the census’s 72 and its $27 + 16 + 29$, the prototype’s counts, the record’s prices — resolve to assertions or printed output of one MSTest suite, published in the papers repository as `labs/paper0A-assembled-verb/`. The dissection’s counts — the 49 author-written tests passing over the freed pieces, and the per-rule hole counts — resolve to the dissection’s audited history: two self-contained git bundles in `paper0A-assets/dissection/`, replayable as ordinary clones.

Pins. The engine is the public repository
(`github.com/alvaroNCubo/puppeteer`), cloned as the sibling

../.../puppeteer every lab in that repository uses, at commit 3160e39 — re-pinned, with every file:line anchor re-resolved, at deposit. The corpus is dotnet/eShop at 9b4f943 and kgrzybek/modular-monolith-with-ddd at 91c8ef2, both cited in References — the same commits the dissection bundles are based on, so the binaries the paper composes and the sources it dissects are one corpus, pinned once. Neither corpus repository is redistributed: fetch-corpus.ps1 clones both at the pinned commits and builds the three assemblies the suite references — the two domain repertoires the body composes, plus the seedwork assembly one of them depends on.

Run. ./fetch-corpus.ps1 once, then dotnet test. Twenty-one labs, all green. The suite compiles against the engine’s public surface only — no signing and no internals grant.

The labs the body cites:

lab	§	measures
TheCompositionBecome sACapabilityLab	§2.2–2.3	one definition, two invocations with different arguments, replay re-performs both from the one record
WhoDecidesWhatCounts AsHistoryLab	§2.4	one body, one verb, three performances: query 2→2, command 2→3 (journalled, replays), query again 3→3 — modality attributed per performance
TheCriterionForConst itutionLab (+ Process ManagerHarness)	§3	the event-sourced coordination arm (the corpus’s shape, author-built and labelled): 0/no · description 6/no (nothing to replay) · constitution 6/yes

lab	§	measures
TheEventSourcedCoordinatorOnAThirdPartyEngineLab	§3	the event-sourced coordinator on Orleans (in-process TestingHost, log-storage provider): 6 transitions and 0 operations in its own journal; reconstruction restores position and re-performs nothing
TheCoordinationArmOnAThirdPartyEngineLab	§3, §7.2	the coordination arm on the Durable Task Framework (in-memory emulator; history via its public dispatch middleware): 6 invocations in the engine's own record; orchestrator code replayed in 6 episodes; no operation re-performed
RepertoireBoundaryLab (+ RepertoireBoundaryFixture/)	§4.2	the compiler refuses the host (CS0122, obtained by compiling a program written to fail); the assembler reaches the internal piece
RetrievalIsALookupLab (+ SubscriptionPaymentBridge)	§4.3	the licensed mint, and nothing about capture: the assembler's key stable across replay, the domain-minted identity fresh, retrieval a lookup in the subject's own state; the minting site marked R_99
ALongTrajectoryOverTwoUntouchedRepertoiresLab	§5.2	17 operations, 4 aggregates, 2 shipped assemblies sharing no type, 0 doubles; 9 events (corpus-emitted, author-composed scenario), none naming the trajectory

lab	§	measures
WhatTheCarrierCarriesLab	§6	one act carried three ways delivers identical values; only the repertoire-minted identity differs
EveryNameStaysWithinOneSubjectsVocabularyLab	§6	the census under two coding rules: population 27 + 16 + 29 = 72 reproduced mechanically; rule one (2 aggregates in one name) codes 0 T; rule two hands its blind spot — zero-aggregate names, process words — to manual coding: 3 residuals, each published with its reason, all S; agreement 72/72 (§6 on what that agreement is not)
TheTrajectoryBecomesObservableLab	§8	two reactions over unrelated patterns answer one question identically, correlated by the recorded act
TheConstitutionPrototypedOnAThirdPartyEngineLab	§3.2, §8	the constitution arrangement prototyped on Orleans, the coordination row's engine: 1 definition and 2 exercises in the engine's own journal; a query performance leaves it unmoved; reconstruction re-executes every journaled statement ; the domain-minted identity differs per replay

lab	§	measures
WhereReplayEquivalenceBreaksLab	§4.3, §8, App. C	the calculus’s prediction, put to the engine: a minted identity captured into a later act’s recorded arguments breaks the composition’s invariant on replay (true→ false), while the same composition referring to it by a supplied key holds (true→true) — the mint fresh in both arms. Order: derivation 0001735 (15:08), lab 0813fec (17:06), same day
WhatTheRecordCostsLab	§8	the record’s price, Release, n=10 after a discarded warm-up, median [min..max]: 554 bytes for definition + first exercise, ~ 112 per additional (exact); replay 4.6 ms at 100 and 15.0 ms at 1,000 exercises; ~ 2,490/s through the plane vs ~ 529,000/s direct — ~ 210x ; published as data/paper0A-assembled-verb/price-release.log

The dissection. The bundles replay as ordinary clones (`git clone eshop-dissection.bundle`); `git log --reverse` lists the pre-registered rules in the order applied with each step’s counts in its commit message, and `git blame` on any surviving line names the rule that spared it. The rule file — with its amendments, its stopping rule, its unequal-objectivity declaration and the withdrawn R7 — is also copied out beside the bundles for direct reading.

Additional instruments retained in the suite but not cited by the body are listed in the lab’s README; they guard the same corpus the cited labs run against.

Appendix B — The Search, Reported as Method

The claim this appendix underwrites does not rest on any search having been exhaustive. It is scoped as *absence over the examined set*, and §3's two questions with §7's battery are a finite decision procedure for any candidate beyond it — a form that survives the discovery of a further neighbour, which matters below, because one of this search's channels is of a kind whose coverage cannot be characterized. The search is reported anyway, in the light form of Kitchenham and Charters (2007), because a claim of the form *not found said together* is a claim about a search, and an instrument must be reported — including what it is not: **this was not a systematic literature review**.

Objective. Find a construction holding the conjunction of §7.7: a capability of a subject, constituted from repertoires the subject does not own, whose definition and exercises survive into the subject's record, with modality attributed at assembly.

Sources, with dates.

- *General web search.* Discovery strings, verbatim, run on 15 and 17 August 2026 — the claim being scoped to what was examined, not to when the examining stopped: `composite command pattern macro command first-class reusable composition object-oriented design; Dietz DEMO enterprise ontology actor role competence authority responsibility transaction pattern production act; event sourcing "command definition" recorded journal replay "invocation" reuse parametrized action definition once invoked many`. Verification strings — used to confirm candidates' bibliographic facts, not to discover — covered the 1972 macro-operators paper, the options framework, Talents, and durable functions. One further string was seeded with its intended answer, is recorded here as biased by construction, and its subject was excluded from the paper on provenance grounds.
- *Open bibliographic APIs* (17 August 2026). **DBLP**, title-level, eight strings: `durable execution workflow` — 2 hits, both 2026 agent-workflow papers, screened out (neither concerns a subject-owned record of a definition and its exercises); `first-class workflow` — 1 hit, workflow-centric research objects (2012), screened out (the first-class citizen is a scholarly artifact, not a capability of a recorded subject); `workflow as code` — 9 hits, screened out (policy-as-code, LLM-assisted workflows, one adoption study; none concerns such a record); `stateful serverless replay, composable capability actor, macro operators planning reuse, action definition journal replay, reified action record subject` — 0 hits each. Three of those five are this paper's own coinages, and a zero at title level is evidence about the string, not about the literature: the zeros are recorded as weak instruments, not as absences. **OpenAlex** — which indexes the metadata of the paywalled venues, nar-

rowing the limitation declared below — the same eight strings, relevance-ranked, top six screened per string: the pass surfaced Burckhardt et al. (2021) at or near the top of two strings, independently reaching the family the adversarial-review channel had contributed, which bounds the earlier miss; the remaining top results belong to families already dispositioned in §7 (the workflow-patterns literature, the Actors model, macro-operators) plus one family screened out here: deterministic record-and-replay for debugging, which records one execution’s nondeterminism so that run can be reproduced — an account of a single run, with no definition, no reuse, and no subject owning the record. **arXiv**, three phrase strings — 0 entries each. No candidate holding the conjunction surfaced in any row.

- *Paywalled indexes.* ACM DL, IEEE Xplore and Scopus were not directly searchable from the execution environment; their metadata is covered by the OpenAlex pass above. The residual limitation is full-text search within those venues.
- *Adversarial review by large language models.* Successive drafts were submitted to large language models instructed to act as program-committee reviewers for a software-engineering venue, across review rounds between 15 and 18 August 2026, each round given the current draft and, where applicable, its prior reviews; the model identities and versions are recorded at deposit alongside the archival snapshots. This channel contributed **seven neighbour families** the string-based rounds above had not surfaced: Talents, the workflow definition/case separation, and algebraic effects; and, in later rounds, durable execution (the Durable Functions semantics with its Temporal/Cadence practice), process mining, multi-stage programming, and computational reflection with the metaobject protocol. The channel’s standing has to be stated exactly, because it is easy to overstate: **it is neither corroboration nor expertise**. Sessions of systems of this kind share training corpora and therefore share blind spots, so agreement between them evidences nothing; their coverage of any literature is unknown and unmeasurable; no output is reproducible across runs; and none can be cited as authority for any claim. Every contributed family was verified by the author against primary sources before entering §7, and the bibliographic facts of each citation were confirmed against the publishers’ registers — nothing in this paper rests on a model’s assertion. What the channel does evidence is the **incompleteness of the string-based rounds**, and that is why it is reported rather than absorbed: seven families the strings did not reach are seven measurements of what the strings’ vocabulary did not cover. A verification pass then located the deficiency: under each field’s own vocabulary — one title-level string per family — the index instrument alone reaches **all seven** (the Talents article itself, the Durable Functions paper itself, and hit counts from 23 to 199 for the rest). The misses were the first round’s vocabulary, not the instrument’s reach, and both facts are retained here as the measurements they are.

Inclusion / exclusion. A candidate entered examination if it plausibly held *any* component of the conjunction; it was then put to the battery of §7. Name-only matches — systems merely called “capability”, “verb” or “assembly” — were excluded.

Examined set and disposition. Sixteen neighbour families, each dispositioned in §7 or where it arises: procedural abstraction; Command and Macro-Command; transaction script and the application service; workflow definition/case; algebraic effects; multi-stage programming; macro-operators; options; macro-by-demonstration and trace JITs, with procedural reflection, the metaobject protocol and prototype languages; Talents; command sourcing; durable execution, with its Orleans ancestry and its Temporal/Cadence practice; the process manager and saga; process mining; counts-as, DEMO and business artifacts; and access audit logging, dispositioned where it arises (§2.4).

Snowballing. Backward from the anchor citations (Gamma et al.; Fowler; Fikes et al.; Sutton et al.). Forward from Orleans (Bykov et al. 2011) to the Temporal practitioner discussion and Bernhardt (2021). DEMO entered from this series’ prior related-work notes.

Threats. Not systematic; a single screener; web ranking is not reproducible; relevance-ranked sources were screened to their top six results per string; full text behind the paywalled venues was not searched; grey literature was reached only through the practitioner sources cited; a zero-hit string measures its own distance from the field’s vocabulary, not any absence in the field; and seven neighbour families entered through an adversarial-review channel whose coverage cannot be characterized and whose outputs are not reproducible — their absence from the string-based rounds is reported as a measured deficiency of those rounds, located by the verification pass in the first round’s vocabulary, and no agreement between sessions of that channel is treated as corroboration. Each of these argues for the scoped form of the claim, and none is argued away.

Appendix C — The Separation, Stated Over a Small Object

§2.4’s four questions — what is computed, what the subject can do, what it does, what counts as its history — are distinctions, and §2 and §3 measure them on running arrangements. A distinction, however, is *stated over an object*, and the smallest object that can carry these four is worth writing down. This appendix gives it: a core calculus, in the structural style of Plotkin (2004), just large enough that assembly, exercise, modality attribution and replay have operational rules — and small enough that each load-bearing separation is a short proposition, with what is merely book-keeping labelled an observation. The labs of Appendix A are its conformance checks, one per claim: `TheCompositionBecomesACapabilityLab` for Observations 1 and 2, `WhoDecidesWhatCountsAsHistoryLab` for Proposition 3, `RetrievalIsALookupLab` with `WhereReplayEquivalenceBreaksLab` for Proposition 4 and its boundary, and the two coordination arms with

TheCriterionForConstitutionLab for Proposition 5.

Syntax. A sequence $x_1..x_n$ is written x^* . Two disjoint name spaces are assumed: operation names f belong to repertoires the subject does not own (the set Op); verb names V are the subject's own. The partition takes the subject as given, deliberately: the calculus carries the separations *given* a subject, and derives no subject from them (§8). That a verb name is, in the engine, an anonymous integer rather than a word (§2.2) changes nothing here — the calculus needs only that the name denotes.

$v ::=$	<code>literal n</code>	values; n a name
$e ::=$	<code>v x</code>	expressions
$s ::=$	<code>x := f(e*) f(e*)</code>	statements over the repertoires
$b ::=$	<code>s ; ... ; s</code>	a body (nonempty)
$m ::=$	<code>cmd qry</code>	modalities
$a ::=$	<code>define V(x*) = b</code>	assembly
	<code> perform V(v*) : m</code>	exercise; m is attributed here

Two scope decisions are made at the grammar and declared as such. Bodies are first-order: no statement invokes a verb, and composition of verbs out of verbs is out of scope here. And the calculus carries no speech of the plane's own — §5.1's `print` and `tell` are outside the object, which covers the composition of operations and not the subject's vocalization; the four questions need no more, and §5.1's claim exceeds this appendix on purpose.

Configurations, and what is given. $\langle D, \sigma, J \rangle$ — definitions $D : Verb \rightarrow (x^*, b)$; domain state σ ; the journal J , a sequence of entries `def(V, x*, b)` and `act(V, v*)`. One constant comes with the calculus: an initial state σ_0 . A configuration is **reachable** when the rules below reach it from $\langle \emptyset, \sigma_0, \varepsilon \rangle$; every claim quantifies over these.

The host as a parameter — axiom (HOST). The repertoires enter the calculus as an interpretation governed by one axiom, named once and cited by name:

(HOST) $\llbracket f \rrbracket$ relates (σ, v^*) to (σ', v, N) , totally on (σ, v^*) , where N is the finite set of fresh names the application mints:
 $N \cap \text{names}(\sigma, v^*) = \emptyset$; $|N|$ is determined by (σ, v^*) and is invariant under bijective renaming of the names in (σ, v^*) ;
 (σ', v) is determined by (σ, v^*) up to bijection on N ; and the relation is equivariant — renaming the names of (σ, v^*) renames the outcome accordingly.

Freshness is the *only* mechanism of the calculus for new names — there is no name supply in the configuration, so nothing re-determinizes the mint. Two applications of the same f to the same arguments agree except in *which* fresh names they drew: which is what the prose of §4.3 says, what the measured factory does, and what makes Proposition 4's bijection necessary rather than decorative. (HOST) is the closure condition §4.3 speaks of, written as an axiom:

whatever an execution depends on either crossed the plane (it is in $\mathbf{v}^*$) or is fresh (it is in $\mathbf{N}$). It is a **closure**, and it is worth saying plainly what it closes out: external I/O, randomness beyond the minting the axiom licenses, foreign mutable state reached behind the plane, callbacks, and clock readings that escape it — §4.3’s undeclared inbounds. One further axiom gives the calculus something to distinguish, and Proposition 5 cites it:

(NONTRIV) there exist f, g in $0p$ and $(\sigma, \mathbf{v}^*)$ whose applications differ in σ' .

Execution. A statement executes under a local environment $\rho : \mathbf{Var} \rightarrow \mathbf{Val}$ — the values flowing between statements, the third constituent §2.2 names — and emits its operation’s result as output:

$$\langle \sigma, \rho \rangle \vdash s \Downarrow \langle \sigma', \rho' \rangle ! v$$

For $x := f(\mathbf{e}^*)$: the arguments $\mathbf{e}^*$ are read under ρ (a variable ρ does not bind is stuck), $\llbracket f \rrbracket$ is applied, minting per (HOST), and $\rho' = \rho[x \mapsto v]$; for bare $f(\mathbf{e}^*)$, $\rho' = \rho$. A body folds its statements left to right, $\langle \sigma, \rho \rangle \vdash b \Downarrow \langle \sigma', \rho' \rangle ! v$, emitting the last statement’s output. The fold always terminates: bodies are finite, invoke no verbs, and $\llbracket f \rrbracket$ is total.

Rules.

$$\begin{array}{c} \text{ops}(b) \subseteq 0p \quad V \notin \text{dom}(D) \\ \hline \langle D, \sigma, J \rangle \text{--define } V(x^*)=b \rightarrow \langle D[V \mapsto (x^*, b)], \sigma, J \cdot \text{def}(V, x^*, b) \rangle \quad (\text{DEFINE}) \\ \\ D(V) = (x^*, b) \quad |v^*| = |x^*| \quad \langle \sigma, [x^* \mapsto v^*] \rangle \vdash b \Downarrow \langle \sigma', \rho' \rangle ! v \\ \hline \langle D, \sigma, J \rangle \text{--perform } V(v^*): \text{cmd} \rightarrow \langle D, \sigma', J \cdot \text{act}(V, v^*) \rangle ! v \quad (\text{PERFORM-CMD}) \\ \\ D(V) = (x^*, b) \quad |v^*| = |x^*| \quad \langle \sigma, [x^* \mapsto v^*] \rangle \vdash b \Downarrow \langle \sigma', \rho' \rangle ! v \\ \hline \langle D, \sigma, J \rangle \text{--perform } V(v^*): \text{qry} \rightarrow \langle D, \sigma, J \rangle ! v \quad (\text{PERFORM-QRY}) \end{array}$$

Two readings of PERFORM-QRY are deliberate. The execution happens — the premise is the same judgment as the command’s, and the output v leaves, which is what keeps a query performance observable at all — and the configuration keeps nothing: not the effect, not the trace. What a query computes is visible at the boundary as its output and durable nowhere. In the engine measured this is literal, because state *is* reconstruction from J (§2.4): what a query changed does not exist after rehydration.

Four idealizations, declared. (1) The define/perform split is primitive; in the engine the two arrive fused in one act and the handler splits them at the reuse seam, and §2.3 measures that it does. (2) PERFORM-QRY is a rollback rule: the premise executes the body and the conclusion restores σ whole, silently undoing whatever a state-changing operation did. That is a substantive modeling decision, and it is **laxer than the engine it models**, which prevents rather than undoes: a query runs under a read lock with the symbol table in

read-only mode, and its parser rejects the statements that would persist (§2.4). The calculus admits a query whose body would change state and unwinds it; the engine refuses to run one. The rules' conclusions agree — the configuration retains nothing either way — and every claim below depends on that conclusion rather than on the path to it, so a proposition proved under the laxer rule holds a fortiori under the stricter engine. What neither reaches is repertoire state held outside σ altogether (statics, caches, external stores), which rolls back nowhere. (3) $\llbracket \mathbf{f} \rrbracket$ is a relation over one store: no aliasing across repertoires, no shared mutable state outside σ , no reentrancy. The shape assumes by construction the tractability of exactly the situation §5.2 exhibits as hard — two repertoires whose only meeting point is the assembler. (HOST) bounds what $\llbracket \mathbf{f} \rrbracket$ may *depend on*; this shape bounds what it may *be*. (4) $\llbracket \mathbf{f} \rrbracket$ is total: no operation fails, so the calculus expresses no failed exercise — misfire and its kin are outside the object.

Replay. $\text{replay}(\mathbf{J})$ is a configuration reached from $\langle \emptyset, \sigma_0, \varepsilon \rangle$ by taking each **def** entry as its DEFINE step and each **act** entry as its PERFORM-CMD step, in $\mathbf{J}$'s order — *a* configuration, not *the*: under (HOST) each run of the replay draws its own fresh names, and the propositions quantify accordingly. For the coordination variant of Proposition 5, $\text{replay}_P(\mathbf{J})$ starts instead from $\langle \mathbf{D}_P, \sigma_0, \varepsilon \rangle$, with $\mathbf{D}_P$ read off the ambient program: the variant's journal holds no **def** entries, so a replay started from $\emptyset$ is stuck at its first **act** — which is itself the point, stated operationally: the record alone does not carry what its replay needs.

The five numbered claims are not of one kind, and the difference is declared rather than left to be discovered. Observations 1 and 2 are **bookkeeping**: they record what the definitions make true — Observation 2 follows from the side condition $V \notin \text{dom}(\mathbf{D})$, a fiat of the calculus — and their value is exactness, not discovery. Propositions 3, 4 and 5 carry the **content**: each could have been false of rules shaped otherwise, and each has an engine measurement that could have disagreed.

Observation 1 (defining computes nothing). DEFINE leaves σ fixed; hence there are reachable configurations equal in σ and different in $\mathbf{D}$. *Why.* Immediate from the rule's conclusion; the witness pair is any configuration and its DEFINE-successor. On the engine: §2.2 exhibits the definition as an entry — content, not effect.

Observation 2 (a capability is not its exercises). After **define** $V(x^*) = \mathbf{b}$ and n command performances with arguments $v^*_1 \dots v^*_n$, $\mathbf{J}$ holds exactly one **def** entry for V and the n entries **act**(V, v^*_i), none containing $\mathbf{b}$ and all referencing the same V . *Why.* DEFINE's side condition $V \notin \text{dom}(\mathbf{D})$ forbids a second definition of V , and PERFORM-CMD appends entries that carry a name and arguments only. The definition/exercise separation is therefore *data in J*, not an interpretation of it. The idealization at work is (1) of the declared four: in the engine, assembly and exercise arrive fused in a single act, and the handler splits them at the reuse seam — a known body is an invocation, a new one a

definition (§2.3). The calculus takes the split as primitive; §2.3 measures that the engine realizes it.

Proposition 3 (what it does is not what counts as its history). Fix $\mathbf{D}$, σ , V , v^* with $\mathbf{D}(V)$ defined and $|v^*|$ matching its parameters. The command and query performances execute under the same premise — one judgment, the same body from the same state, the same output emitted — and differ in journal membership, which is therefore a function of $\mathbf{m}$ alone. And $\mathbf{m}$ is a component of the *performance*, supplied at the exercise rule: it is not computed from the body, the state, or the outcome. The calculus establishes the **location** of the choice, not its author — there is no attribution judgment here, and none is needed for the separation. *Proof.* The rules share their premise and differ only in conclusion; the emitted output is what keeps the query half of the witness observable, since the query’s configuration is unchanged. Who occupies that position in the measured arrangement is §2.4’s question, and its answer there is the assembler. On the engine: §2.4 — the journal head unmoved under the query, advanced under the command, the domain computing the same value both times.

Definition ($\cong$, determination, and what it is for). Two configurations are **equivalent**, $C \cong C'$, when some bijection π relates them componentwise, mapping the names minted during C' ’s trace to those minted during C ’s and fixing every other name — the two traces are distinct, and π is exactly the correspondence between what each of them minted. A pair $(J, \llbracket \cdot \rrbracket)$ **determines** replay’s outcome up to $\cong$ when every configuration any run of $\text{replay}(J)$ reaches falls in one $\cong$ -class. And $\cong$ is what §3.1’s *outcome exists again* means, made precise: **the whole’s outcome exists again exactly when the replayed configuration is $\cong$ to the original** — which is the same clause §3.1 states in its own words as identity up to fresh values, and the reason it must be stated before the criterion is used. Propositions 4 and 5 use exactly these notions. (The local environment of the execution judgment is written ρ ; the bijection here and in what follows is always π .)

Proposition 4 (the record constitutes, up to fresh names — and no closer). Assume (HOST); let $\langle \mathbf{D}, \sigma, J \rangle$ be reachable, and let the trace be **journal-closed**: no entry of J — neither a **def** body nor an **act**’s arguments — contains a name minted during that trace. Names are values in this grammar, so a body can carry one and the hypothesis has to range over the whole journal, not only its arguments; what it says is that everything the record holds crossed the plane from outside, which is exactly what makes π fix J . Then every run of $\text{replay}(J)$ reproduces $\mathbf{D}$ exactly — definitions mint nothing, so that much is a strict equality — reproduces J (by journal-closedness no entry contains a trace-minted name, so π fixes every one of them: $\pi(J) = J$, hence $J_r = J$) — and reaches a state σ_r with a bijection π on minted names such that $\pi(\sigma_r) = \sigma$. The three conclusions together are $\text{replay}(J) \cong \langle \mathbf{D}, \sigma, J \rangle$, which is what Proposition 5 cites. The renaming cannot be dropped: **exact identity is not available for every interpretation satisfying (HOST)**, and equivalence

up to bijective renaming is. The witness is immediate — an interpretation that draws a different fresh name on each application satisfies the axiom and defeats name equality across runs — so §3.1’s clause is proved necessary rather than conceded. No maximality beyond that is claimed, and none is needed. *Proof sketch.* Induction over the define/command sub-trace with hypothesis $\pi_k(\sigma_r^k) = \sigma^k$; query steps change nothing in the configuration, so they drop out. At the next command step both sides execute the same journaled v^* , which by journal-closedness contains no trace-minted name, so π_k fixes it; (HOST)’s equivariance applied to π_k transports the replay’s application at (σ_r^k, v^*) to an application at (σ^k, v^*) — the live one — whose outcomes therefore correspond under an extension $\pi_{\{k+1\}} \supseteq \pi_k$ over the step’s fresh names, equal in number by (HOST)’s count invariance. The extensions compose because fresh names are new on both sides. On the engine: the constitution row of §3.2, whose *yes* is stated under exactly this equivalence (§3.1), and §4.3’s measured identity — fresh on every replay while the supplied key holds — which is π made visible.

Where the hypothesis fails. Let a trace-minted name be *captured* — read by the assembler and passed into a later act’s journaled arguments. The live application’s argument then aliases a name of the live state; the replayed application’s argument is the recorded literal, which aliases nothing in the replayed state, since replay minted fresh; and no bijection can both align the states and fix the literal. Replay equivalence breaks — not at the mint, but at the **capture** of the mint.

The converse is the calculus’s usable half: if every reference to a minted value travels between acts as a lookup under a key that crossed the plane, rather than as the value itself, the trace is journal-closed and Proposition 4 applies. That is a sufficient condition — a characterization of when the guarantee holds, not advice about how to write verbs.

Both halves were then measured, in a lab written after this derivation (§8 for the finding and its standing; Appendix A for the arms, the numbers and the commit order). One clause of that measurement belongs here rather than there: what the lab observes is a composition’s invariant holding live and failing after replay, and that this *is* a violation of $\cong$ is the calculus’s step — a configuration whose invariant fails cannot be related by any renaming to one whose invariant holds, since π maps values to values and cannot make an equality false.

What the hypothesis costs the measured arrangements. Nothing, in all three: §2.2’s flagship is a single act whose arguments are all supplied, so its trace is journal-closed outright; §5.2’s scenario is journaled nowhere, exercising the two repertoires directly to exhibit what neither can say; and §5.1’s co-binding travels inside one body, through the local environment. The hypothesis bites only where the measurement puts it — a value minted inside an operation, read out, and recorded as a later act’s argument — and what would violate (HOST) itself is different again: uncaptured I/O, escaped clocks, foreign mutable state.

The proposition’s scope has to be stated as exactly as its proof: **Proposition 4 characterizes *replayable* constitution under (HOST); it does not characterize constitution itself.** And it is doubly relative: to (HOST), and to the interpretation — replay reads $\llbracket f \rrbracket$ from outside the record, and Proposition 5 makes that second relativization explicit. Where (HOST) fails, what fails is replay equivalence — the (b)-half of §3’s innermost level — and nothing above it: DEFINE still extended $\mathbf{D}$, the definition and its exercises still stand in $\mathbf{J}$, and Observations 1 and 2 and Proposition 3 hold without the axiom, since none of their arguments touches $\llbracket f \rrbracket$. A verb whose repertoire reaches behind the plane in the ways (HOST) closes out is still defined, still exercised, still owned; its record has lost a guarantee, not the verb its existence.

Proposition 5 (what the criterion separates is how much stays ambient). Under Proposition 4’s hypotheses, $(\mathbf{J}, \llbracket \cdot \rrbracket)$ determines constitution’s replay up to $\cong$; coordination’s replay additionally depends on the ambient program $\mathbf{P}$.

Proof. The positive half is Proposition 4, which concludes $\text{replay}(\mathbf{J}) \cong \langle \mathbf{D}, \sigma, \mathbf{J} \rangle$ at the level of whole configurations — the level $\cong$ is defined at — and inherits its hypotheses, journal-closedness included. The negative half needs neither (HOST)’s freshness clauses nor anything about a particular engine: by (NONTRIV) there are operations whose outcomes differ at some state, reachable in the variant since the two calculi share their execution rules and differ only in where a definition lives; so ambient bodies $\mathbf{b} \neq \mathbf{b}'$ differing in such an operation make the same $\mathbf{J}$ replay to different states under $\mathbf{P}[\mathbf{V} \mapsto \mathbf{b}]$ and $\mathbf{P}[\mathbf{V} \mapsto \mathbf{b}']$, the interpretation held fixed.

Interpretation. Neither calculus lets the record settle its replay absolutely — re-performing reads $\llbracket f \rrbracket$, code outside $\mathbf{J}$, in both — which is why §3.1’s (b) holds the interpretation and σ_0 fixed. What separates them is the remainder beyond $(\mathbf{J}, \llbracket \cdot \rrbracket)$: what moves into the record is the *composition*, never the operations, a difference in how much stays ambient rather than a categorical “record alone”, and §8 carries that remainder as a limit. Because the negative half rests only on a $\mathbf{P}$ free to vary, the separation survives any further narrowing of the positive half. On the engine: the durable-execution row of §3.2, where the history holds the exercises and replay re-walks compiled code, so the same history under changed code means something else; and the constitution row, where with the repertoires held fixed, changing nothing but the record is enough. §3.1’s (b) asks this proposition’s question of a record, and its two halves are answered here and at Proposition 4 with the measurement of §3.2.

The four questions, and the one arrow among them. Over a reachable configuration, three of §2.4’s four lines are projections: *what the subject can do* is $\text{dom}(\mathbf{D})$; *what counts as its history* is $\mathbf{J}$; *what is computed* is σ . The fourth — *what the subject does* — is deliberately not in the configuration: performances are the trace’s property, and Proposition 3 witnesses their separation at the rules themselves, one premise under two conclusions. What is claimed is exactly the separations §2.4 needs, each with its witness: equal σ with different

$\mathbf{D}$ (Observation 1), so what is computed does not fix what the subject can do; equal $\mathbf{J}$ with different executions (Proposition 3’s query half), so the history does not reveal everything the subject did; equal $\mathbf{D}$ with different $\mathbf{J}$ (runs of different exercise counts), so the repertoire does not fix the history. And one positive dependency, which the paper uses: under Proposition 4’s hypotheses $\mathbf{J}$ reconstructs $\mathbf{D}$ exactly and σ up to $\cong$. That arrow is the finding rather than the table’s defect — recorded constitution *is* the arrangement in which the history suffices for the rest — and no wider independence is asserted here.

What is not claimed. No metatheory beyond these claims is developed. In particular, no equivalence between a programming model and its record-replay implementation is proven — the durable-execution neighbour proves exactly such a theorem for its model (Burckhardt et al., 2021), and this appendix’s object is smaller and its aim narrower: to carry the distinctions. The calculus is a characterization of the measured arrangement. Conformance of the engine to the rules is checked by the labs, one per claim, and checked is all it is.

References

Entries without a refereed venue — practitioner forum posts — are given with their access date and are marked as such; an archival snapshot is pinned at deposit.

The letter suffixes on this series’ own entries denote the paper’s number in the series, not the order of appearance here — 2026c is Paper 3, 2026d Paper 4, 2026h Paper 8, 2026i Paper 9 — so that a given paper carries the same label wherever in the series it is cited. In the blind review copy the four series entries and the *Dependencies* note are replaced by anonymised placeholders.

Austin, J. L. (1962). *How to do things with words*. Oxford University Press.

Bala, V., Duesterwald, E., & Banerjia, S. (2000). Dynamo: A transparent dynamic optimization system. *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2000)*, 1–12.

Bauer, A., & Pretnar, M. (2015). Programming with algebraic effects and handlers. *Journal of Logical and Algebraic Methods in Programming*, 84(1), 108–123.

Bernhardt, M. (2021). *Tour of Temporal: Welcome to the workflow* [Blog post; practitioner source, no refereed venue]. <https://manuel.bernhardt.io/2021/04/12/tour-of-temporal-welcome-to-the-workflow/> (accessed 17 August 2026; archival snapshot to be pinned at deposit).

Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., & Meiklejohn, C. S. (2021). Durable functions: Semantics for stateful serverless. *Proceedings of the ACM on Programming Languages*, 5(OOPSLA), Article 133. <https://doi.org/10.1145/3485510>

- Bykov, S., & Fateev, M. (2021). *Temporal vs Akka and Lagom* [Forum discussion; practitioner source, no refereed venue]. Temporal Community Forum. <https://community.temporal.io/t/temporal-vs-akka-and-lagom/2589> (accessed 17 August 2026; archival snapshot to be pinned at deposit).
- Bykov, S., Geller, A., Kliot, G., Larus, J. R., Pandya, R., & Thelin, J. (2011). Orleans: Cloud computing for everyone. *Proceedings of the 2nd ACM Symposium on Cloud Computing (SoCC 2011)*, Article 16.
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.
- Cypher, A. (Ed.). (1993). *Watch what I do: Programming by demonstration*. MIT Press.
- Dietz, J. L. G. (2006). *Enterprise ontology: Theory and methodology*. Springer.
- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- Fikes, R. E., Hart, P. E., & Nilsson, N. J. (1972). Learning and executing generalized robot plans. *Artificial Intelligence*, 3(4), 251–288.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Freire, J., Koop, D., Santos, E., & Silva, C. T. (2008). Provenance for computational tasks: A survey. *Computing in Science & Engineering*, 10(3), 11–21.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Garcia-Molina, H., & Salem, K. (1987). Sagas. *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 249–259.
- Gregor, S. (2006). The nature of theory in information systems. *MIS Quarterly*, 30(3), 611–642.
- Grzybek, K. (2024). *Modular monolith with DDD* [Computer software]. GitHub. <https://github.com/kgrzybek/modular-monolith-with-ddd> (commit 91c8ef2, accessed 17 August 2026).
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.
- Hewitt, C., Bishop, P., & Steiger, R. (1973). A universal modular ACTOR formalism for artificial intelligence. *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI 1973)*, 235–245.
- Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.
- Jones, A. J. I., & Sergot, M. (1996). A formal characterisation of institutionalised power. *Logic Journal of the IGPL*, 4(3), 427–443.

- Kiczales, G., des Rivières, J., & Bobrow, D. G. (1991). *The art of the metaobject protocol*. MIT Press.
- Kitchenham, B., & Charters, S. (2007). *Guidelines for performing systematic literature reviews in software engineering* (Technical Report EBSE-2007-01). Keele University and Durham University.
- LeBlanc, T. J., & Mellor-Crummey, J. M. (1987). Debugging parallel programs with Instant Replay. *IEEE Transactions on Computers*, C-36(4), 471–482.
- Maes, P. (1987). Concepts and experiments in computational reflection. *Proceedings of the ACM Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA 1987)*, 147–155.
- Meyer, B. (1997). *Object-oriented software construction* (2nd ed.). Prentice Hall.
- Moreau, L., & Missier, P. (Eds.). (2013). *PROV-DM: The PROV data model* (W3C Recommendation). World Wide Web Consortium.
- .NET Foundation. (2026). *eShop: A reference .NET application* [Computer software]. GitHub. <https://github.com/dotnet/eShop> (commit **9b4f943**, accessed 17 August 2026).
- Nigam, A., & Caswell, N. S. (2003). Business artifacts: An approach to operational specification. *IBM Systems Journal*, 42(3), 428–445.
- O’Callahan, R., Jones, C., Froyd, N., Huey, K., Noll, A., & Partush, N. (2017). Engineering record and replay for deployability. *Proceedings of the USENIX Annual Technical Conference (ATC 2017)*, 377–389.
- PCI Security Standards Council. (2022). *Payment Card Industry Data Security Standard: Requirements and testing procedures* (v4.0), Requirement 10.
- Peffer, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77.
- Plotkin, G. D. (2004). A structural approach to operational semantics. *Journal of Logic and Algebraic Programming*, 60–61, 17–139.
- Ressia, J., Gîrba, T., Nierstrasz, O., Perin, F., & Renggli, L. (2014). Talents: An environment for dynamically composing units of reuse. *Software: Practice and Experience*, 44(4), 413–432. <https://doi.org/10.1002/spe.2160>
- Rivera, A. (2026c). Reactions and the partition: opt-in eventual consistency in actor-native systems. *Puppeteer Papers Series*, Paper 3 [Preprint]. Zenodo. <https://doi.org/10.5281/zenodo.20792156>
- Rivera, A. (2026d). Preserving semantic continuity across actors: a tell-based approach without orchestration. *Puppeteer Papers Series*, Paper 4 [Preprint]. Zenodo. <https://doi.org/10.5281/zenodo.21207062>

- Rivera, A. (2026h). Inference without Authority: the three authorities that govern an output. *Puppeteer Papers Series*, Paper 8 [Preprint]. Zenodo. <https://doi.org/10.5281/zenodo.21499637>
- Rivera, A. (2026i). Identity precedes staging: one play, many stages. *Puppeteer Papers Series*, Paper 9 [Preprint]. Zenodo. <https://doi.org/10.5281/zenodo.21894206>
- Searle, J. R. (1969). *Speech acts: An essay in the philosophy of language*. Cambridge University Press.
- Searle, J. R. (1995). *The construction of social reality*. Free Press.
- Shaw, M. (2003). Writing good software engineering research papers. *Proceedings of the 25th International Conference on Software Engineering (ICSE 2003)*, 726–736.
- Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., & Shanbhag, C. (2010). *Dapper, a large-scale distributed systems tracing infrastructure* (Technical Report dapper-2010-1). Google.
- Smith, B. C. (1984). Reflection and semantics in Lisp. *Proceedings of the 11th ACM Symposium on Principles of Programming Languages (POPL 1984)*, 23–35.
- Stol, K.-J., & Fitzgerald, B. (2018). The ABC of software engineering research. *ACM Transactions on Software Engineering and Methodology*, 27(3), 11:1–11:51.
- Sutton, R. S., Precup, D., & Singh, S. (1999). Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2), 181–211.
- Taha, W., & Sheard, T. (1997). Multi-stage programming with explicit annotations. *Proceedings of the ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation (PEPM 1997)*, 203–217.
- Ungar, D., & Smith, R. B. (1987). Self: The power of simplicity. *Proceedings of the ACM Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA 1987)*, 227–242.
- U.S. Department of Health and Human Services. (2003). Security standards for the protection of electronic protected health information: Audit controls. 45 C.F.R. § 164.312(b).
- van der Aalst, W. M. P. (2016). *Process mining: Data science in action* (2nd ed.). Springer.
- van der Aalst, W. M. P., & van Hee, K. (2002). *Workflow management: Models, methods, and systems*. MIT Press.